Machine Learning · Advanced Ensembles

Advanced Gradient Boosting

XGBoost, LightGBM and CatBoost: Principles and 2026 Implementation

Follow the principles by hand; leave the coding to Claude Code

StatQuest examples reproduce, step by step, how the three engines build trees,
and the lecture closes with prompts that hand implementation to Claude Code and coding cautions (versions, aliases).

Three messages of this lecture

The three engines attack different bottlenecks on the same boosting skeleton

1

XGBoost designs trees directly with a second-order approximation and regularization

It computes similarity and gain from the sum and count of residuals; λ and γ control tree size and output values.

2

LightGBM finds the same splits while looking at less data

Leaf-wise growth cuts the loss quickly, and GOSS and EFB reduce the rows and columns to compute.

3

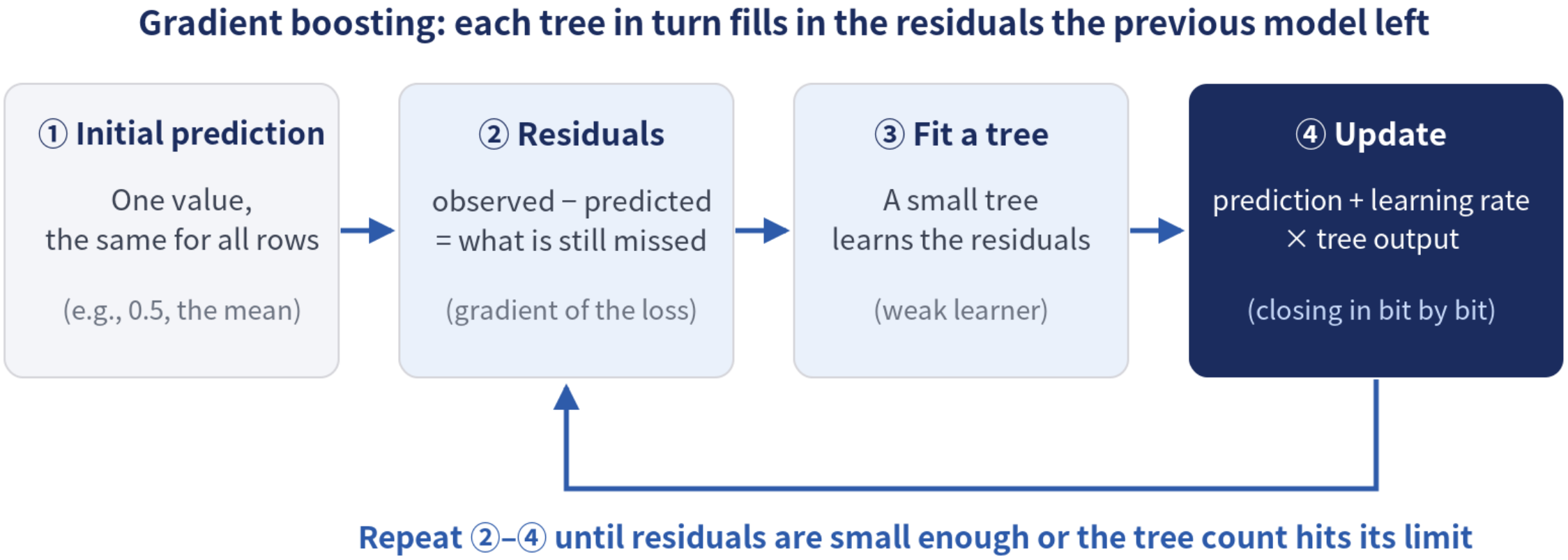
CatBoost keeps order to prevent target leakage

Ordered Target Encoding and Ordered Boosting reduce the overfitting that categorical features cause.

Accuracy differs little across the three engines. Choose by data structure and speed needs.

Boosting is an ensemble that fills in residuals in sequence

The four steps of gradient boosting: all three engines in this lecture share this skeleton



Three engines, one skeleton, different bottlenecks

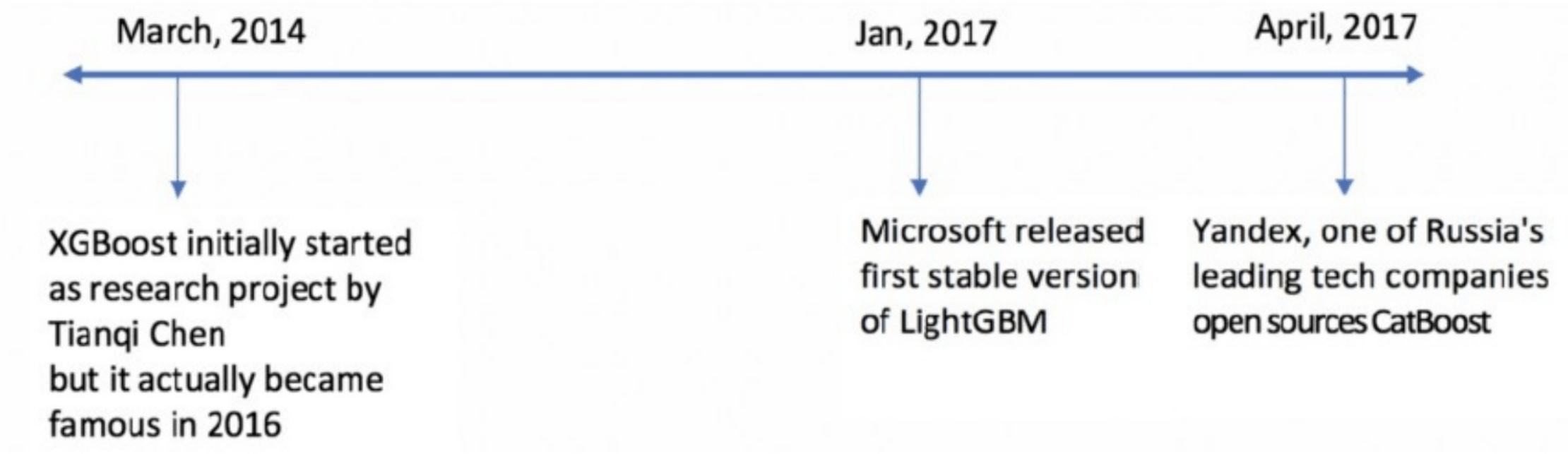
Background and core ideas compared; versions as of November 2026

Item	XGBoost	LightGBM	CatBoost
Developer	Tianqi Chen (research began in 2014)	Microsoft (released January 2017)	Yandex (released April 2017)
Tree growth	Grows evenly, level by level	Grows deep from the highest-loss leaf	Uses the same split at the same depth
Core technique	Computes split gain with a second-order approximation	Keeps only large gradients and bundles features	Keeps order to prevent target leakage
Categorical handling	Available with enable_categorical on	Auto-detects the category dtype	Built in by default; the strongest
Latest version	3.4.1 (August 2026)	4.7.0 (July 2026)	1.2.10 (February 2026)

Blending all three models is now the standard combination in competitions.

Two engines followed the path XGBoost opened within a year

Timeline: XGBoost became widely known through Kaggle wins in 2016



01

PART 1 · XGBoost

XGBoost: Regularized Second-Order Boosting

See the objective, build one regression tree by hand, then look at the devices for large data

XGBoost delivered both accuracy and system optimization

The leading GBDT before LightGBM, and still widely used in industry and competitions

Six strengths

- Regularization in the objective prevents overfitting.
- Learns which way missing values should go.
- Builds trees in parallel.
- Handles large data with cache and disk awareness.
- Has built-in cross-validation.

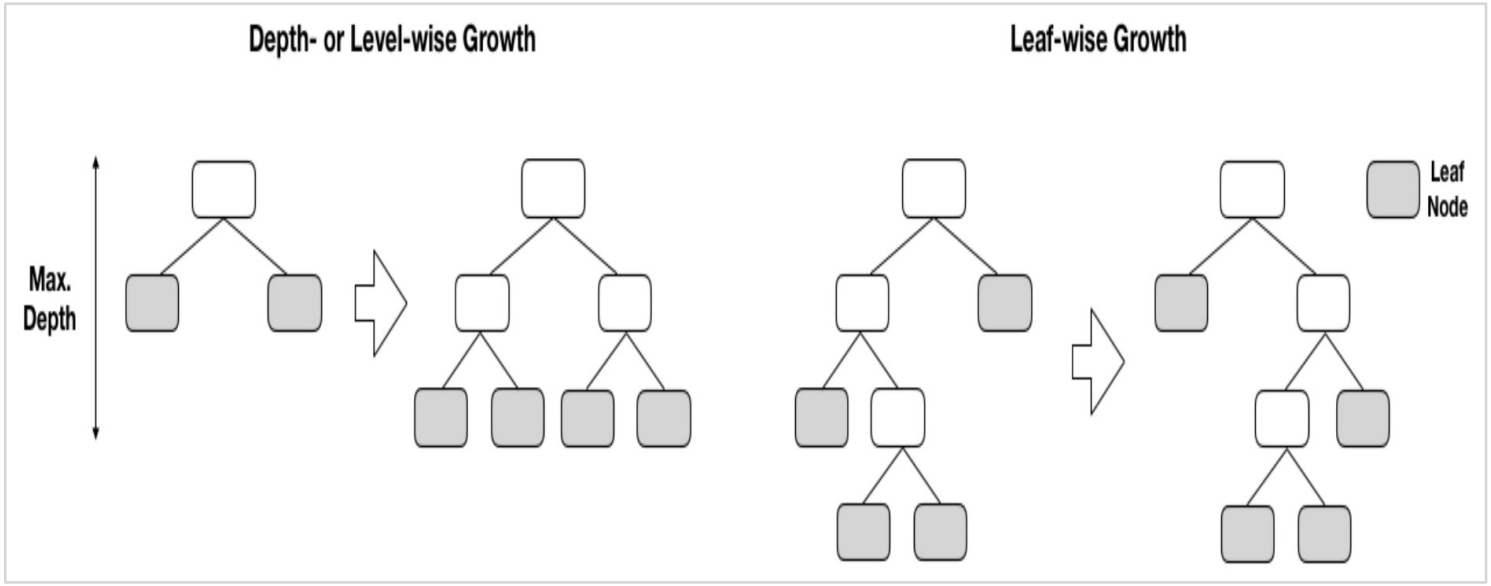


XGBoost trees grow evenly, level by level

Depth-wise (level-wise) growth: the tree does not deepen on one side only

How to read it

- Left: all nodes at one depth split before moving to the next depth.
- Right: LightGBM's leaf-wise growth, revisited in PART 2.
- Level-wise growth is slower but carries less overfitting risk.



Objective = prediction loss + tree complexity penalty

Regularized objective from the original lecture material (Chen & Guestrin, 2016)

$$\mathcal{L}(\phi) = \underbrace{\sum_i l(\hat{y}_i, y_i)}_{\text{prediction loss}} + \underbrace{\sum_k \Omega(f_k)}_{\text{tree complexity penalty}}$$

$$\Omega(f) = \underbrace{\gamma T}_{\text{leaf count penalty}} + \underbrace{\frac{1}{2}\lambda \|w\|^2}_{\text{leaf output size penalty}}$$

Role of γ (number of leaves)

- Each extra leaf costs a penalty of γ.

Role of λ (leaf output)

- Larger leaf outputs incur larger penalties.

Split scores come from sums of gradients and Hessians

Split score from a second-order Taylor approximation: split points are derived directly from the loss function

$$\mathcal{L}_{split} = \frac{1}{2}\left[\frac{(\sum_{i\in I_L} g_i)^2}{\sum_{i\in I_L} h_i+\lambda} + \frac{(\sum_{i\in I_R} g_i)^2}{\sum_{i\in I_R} h_i+\lambda} - \frac{(\sum_{i\in I} g_i)^2}{\sum_{i\in I} h_i+\lambda}\right] - \gamma$$

Reading the symbols

- g is the first derivative of the loss (gradient); h is the second derivative (Hessian).
- The bracket holds the scores of the left node, right node and pre-split node.

Meaning

- Split only if the left and right scores sum to at least γ more than before the split.
- With squared-error loss, h equals 1 and this matches the similarity formula that follows.

The leaf output value minimizes the approximate loss

The original's "Mathematical Background (Just Look!)": differentiate, set to 0, and the output value appears

Three steps

- ① Differentiate the approximate loss with respect to the leaf output.
- ② Set the derivative to 0.
- ③ Solving gives the gradient sum divided by the sum of the Hessian sum and λ.

To find the O_{value} that minimizes...

$$\left[\sum_{i=1}^{n} L(y_i, p_i^0 + O_{value})\right] + \gamma + \frac{1}{2}\lambda O_{value}^2$$

1) Take the derivative...

$$\frac{d}{dO_{value}}(g_1 + g_2 + g_3)O_{value} + \frac{1}{2}(h_1 + h_2 + h_3 + \lambda)O_{value}^2$$

2) Set to zero...

$$(g_1 + g_2 + g_3) + (h_1 + h_2 + h_3 + \lambda)O_{value} = 0$$

3) Solve for $f_k(x_i)$...

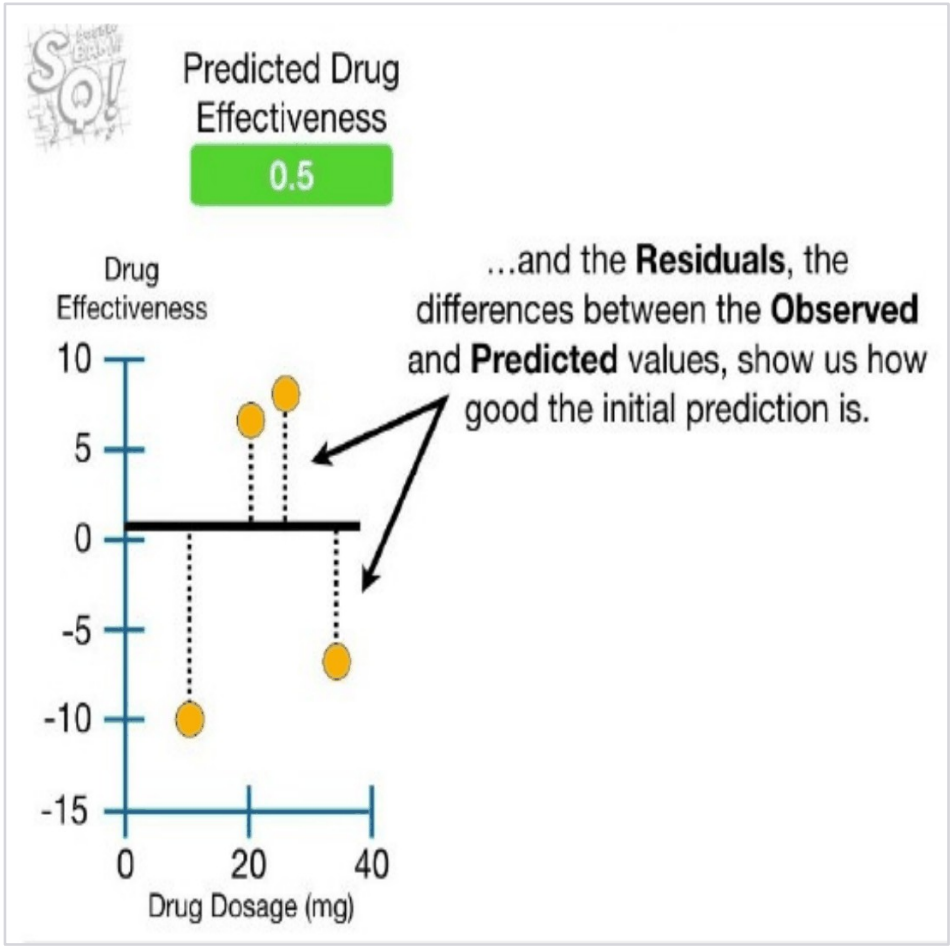
$$O_{value} = \frac{-(g_1 + g_2 + g_3)}{(h_1 + h_2 + h_3 + \lambda)}$$

Start by computing residuals from the first prediction, 0.5

StatQuest example: four rows predicting drug effectiveness from dosage

Starting point

- An XGBoost regression tree has three elements: boosting, regularization and a unique regression tree.
- The initial prediction defaults to 0.5.
- The residuals are −10.5, 6.5, 7.5 and −7.5.

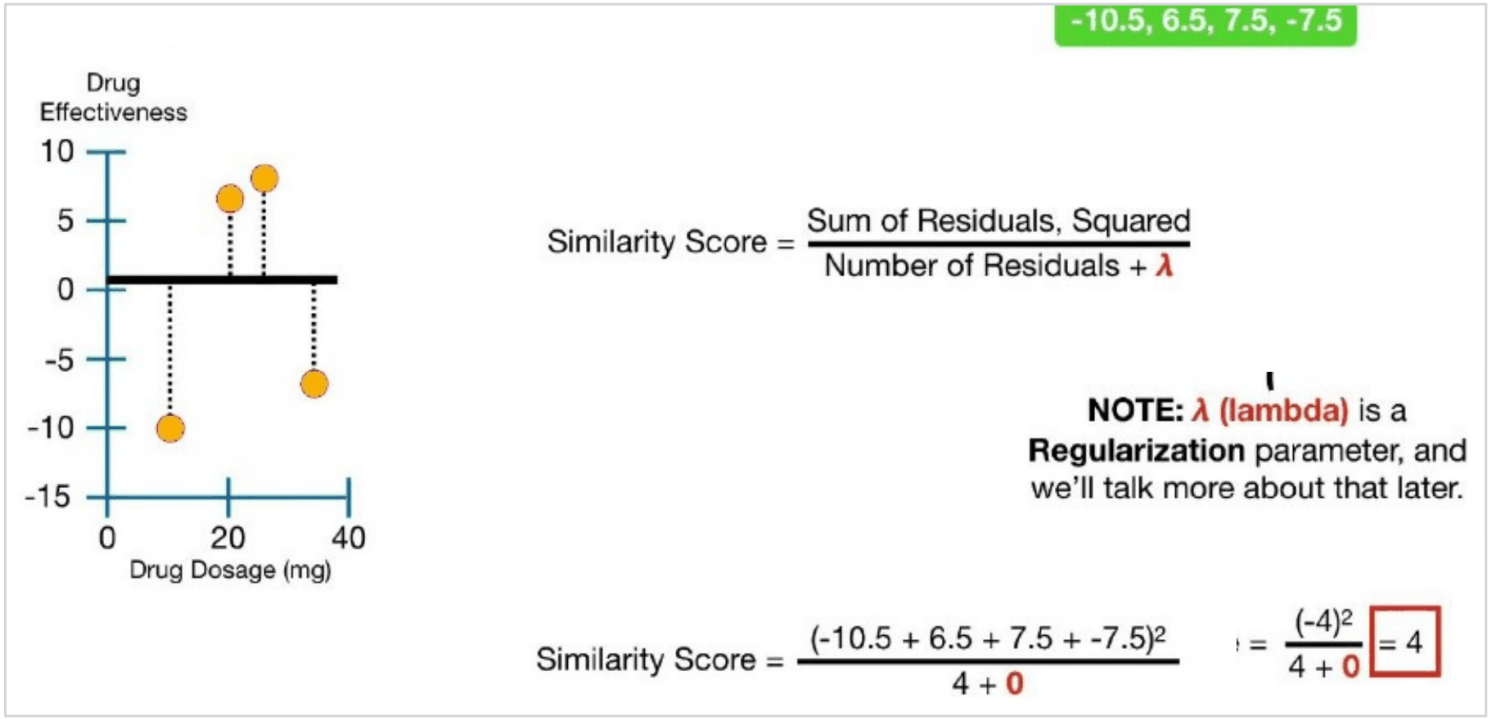


Put all residuals in one leaf and compute similarity

Set λ to 0 and find the similarity of the root node

How to read it

- All four residuals sit in the root leaf.
- Residuals of opposite sign cancel, so similarity is a small 4.
- λ is the regularization parameter; its effect comes later.

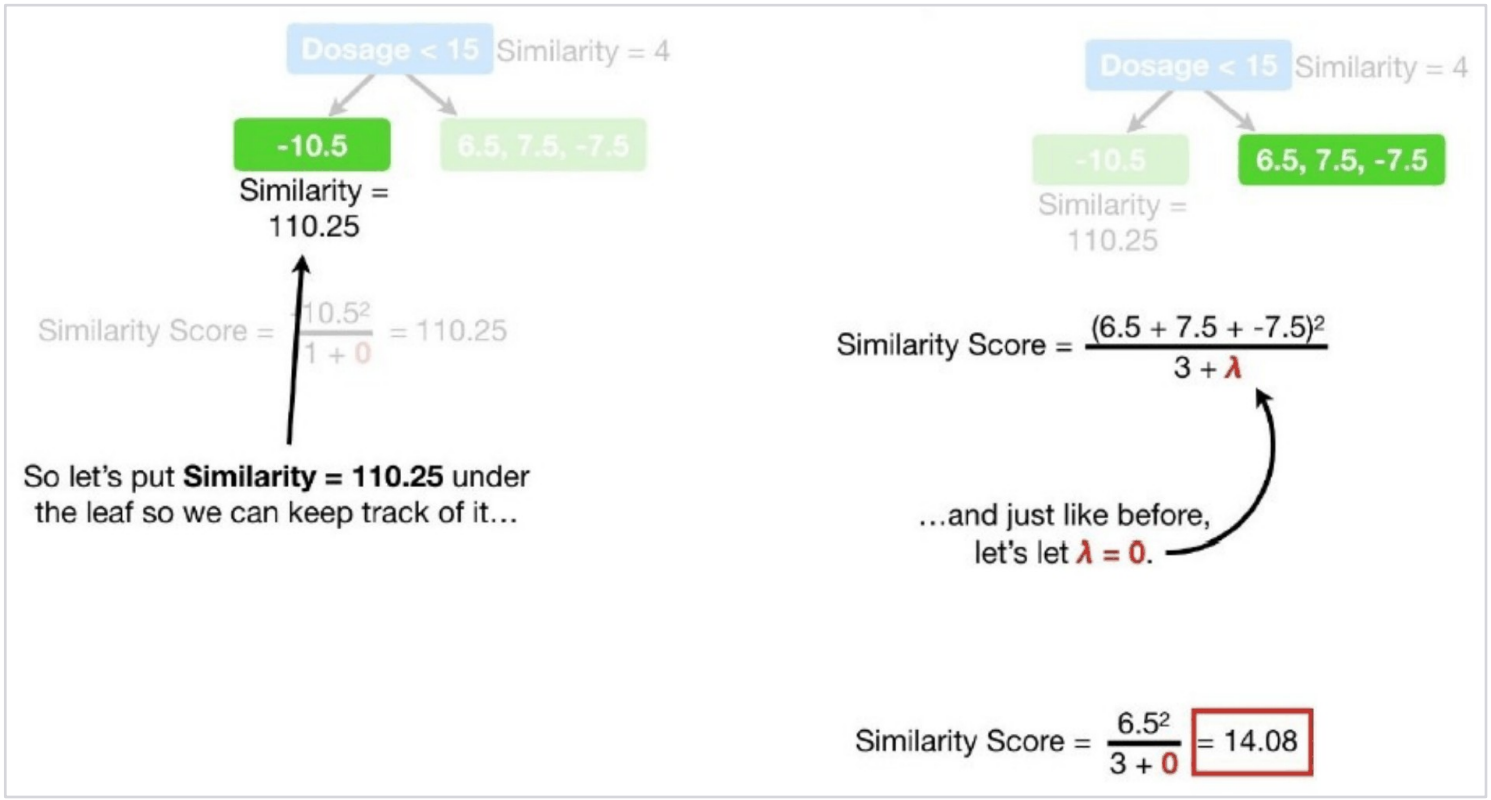


The first split candidate is Dosage < 15

Use the average of two adjacent rows as the threshold and compute similarity on both sides

How to read it

- Only residual −10.5 stays on the left, so similarity jumps to 110.25.
- On the right, 6.5, 7.5 and −7.5 cancel, giving 14.08.
- The more alike the grouped residuals, the larger the similarity.

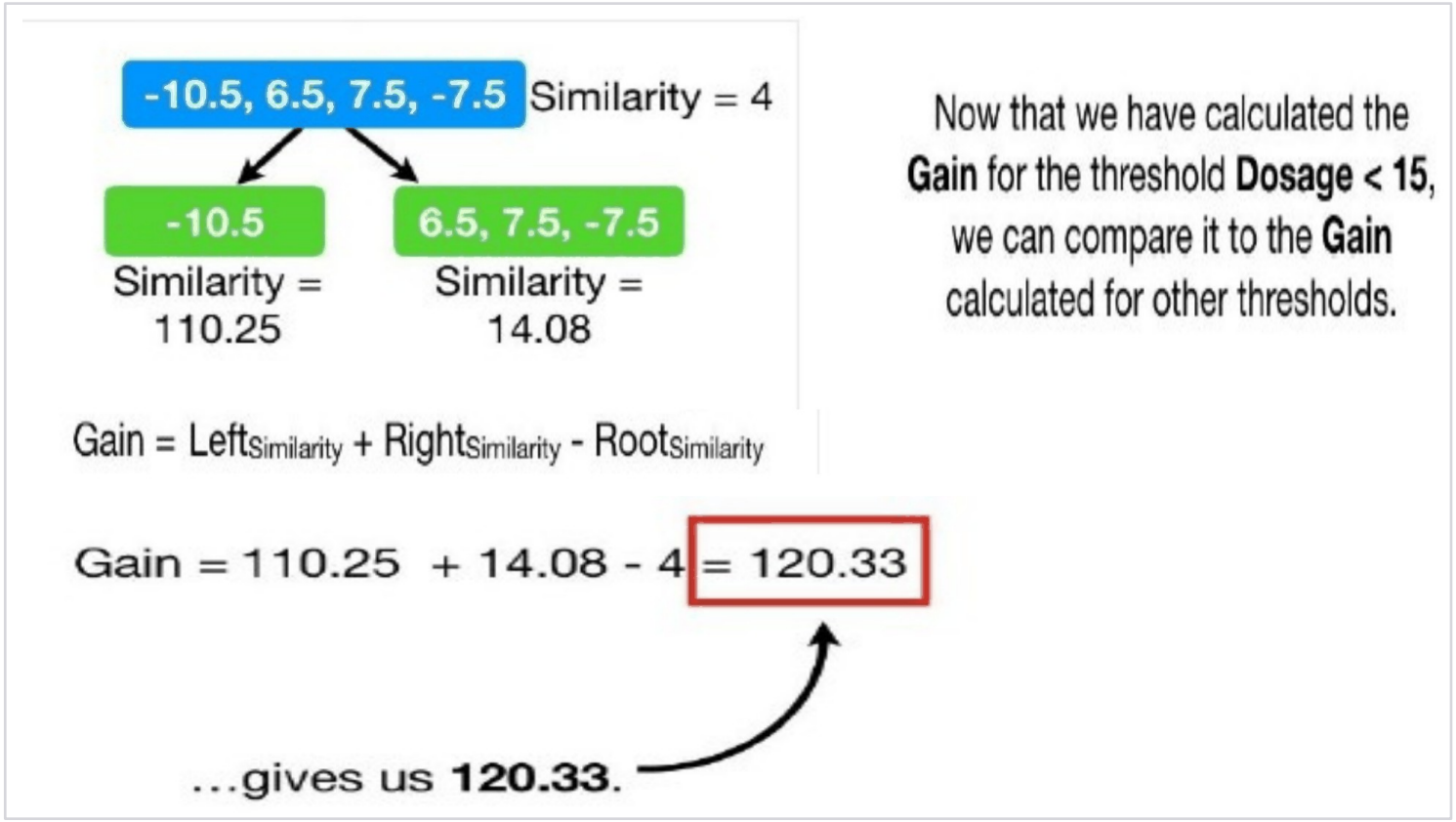


Gain measures the value of a split

The gain of Dosage < 15 is 110.25 + 14.08 − 4 = 120.33

How to read it

- Gain is left similarity plus right similarity minus root similarity.
- Compute the gain of other thresholds the same way and compare.

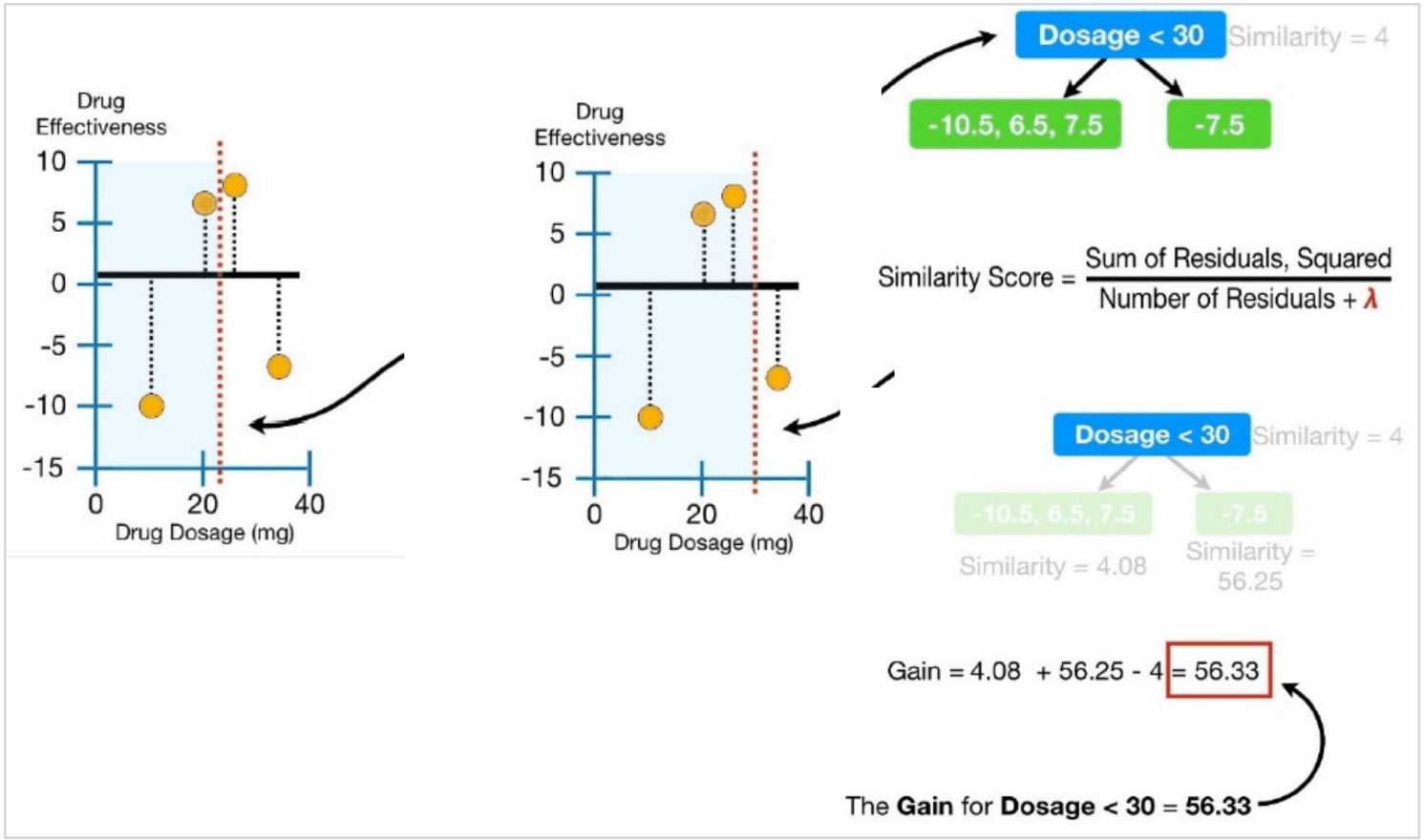


The threshold with the largest gain becomes the first split

Dosage < 22.5 has gain 4 and Dosage < 30 has 56.33, both below Dosage < 15

How to read it

- Shift the threshold one step at a time and recompute the gain.
- Of the three candidates, Dosage < 15 has the largest gain (120.33).
- Trying every candidate like this is called a greedy algorithm.

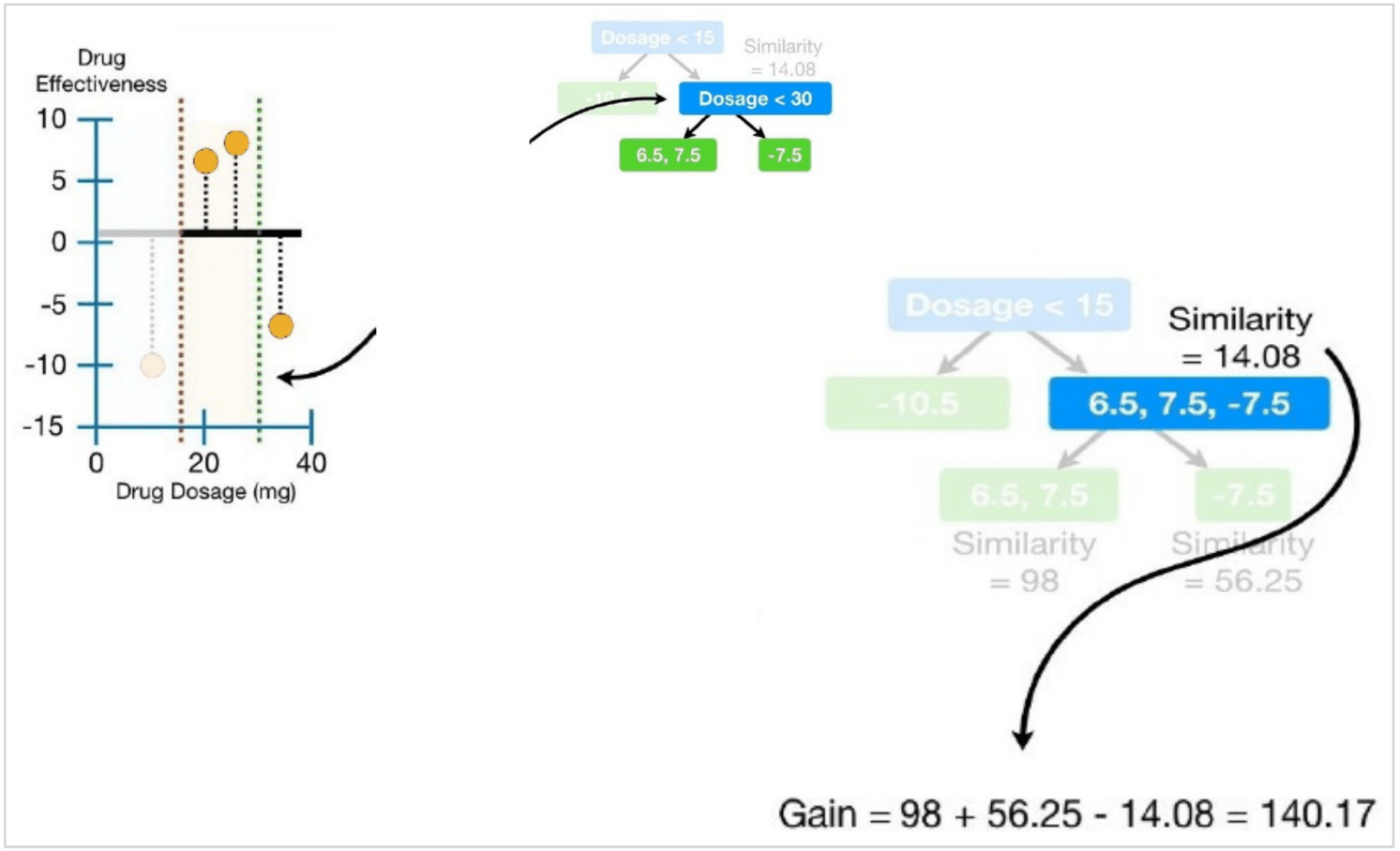


Split the right leaf once more the same way

The left leaf holds a single residual and cannot be split further

How to read it

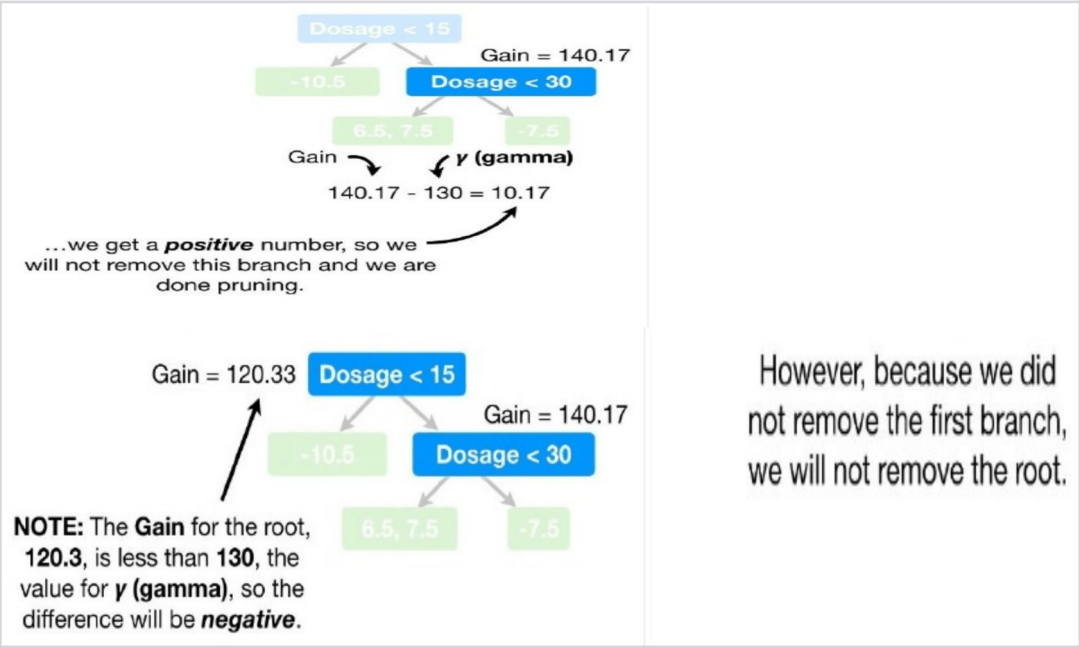
- Try the right residuals 6.5, 7.5, −7.5 at two thresholds.
- Dosage < 30 has gain 140.17, larger than Dosage < 22.5 (28.17).
- The example limits depth to 2; the default is 6.



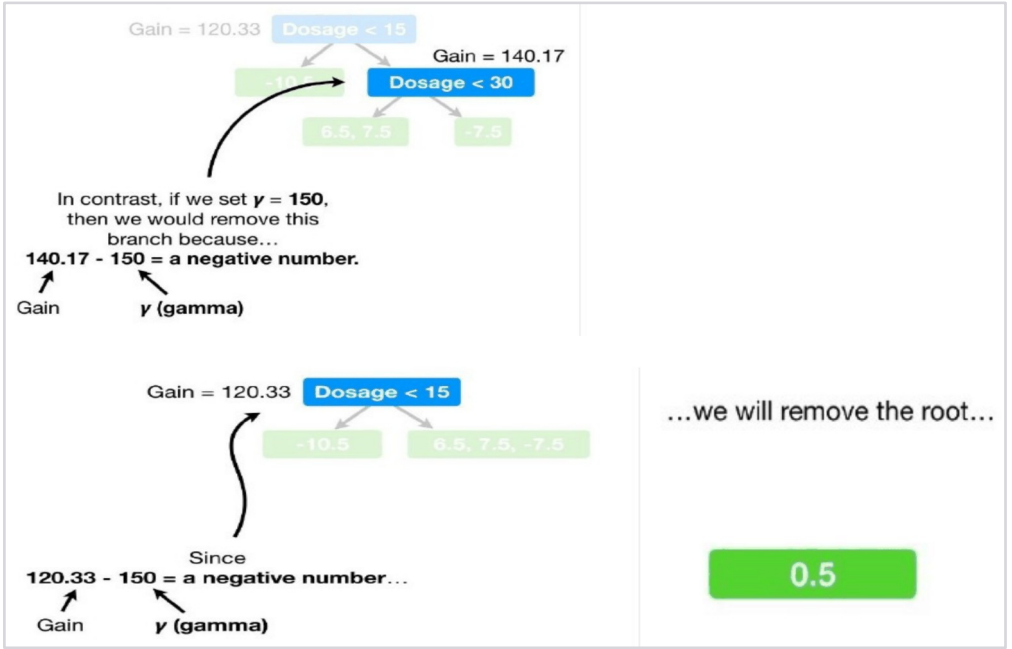
Branches with gain below γ are pruned

Pruning starts from the lowest branch and uses the sign of "gain – γ"

With γ = 130, the lower branch's gain of 140.17 is larger, so the branch stays



With γ = 150, both the lower branch and the root are pruned



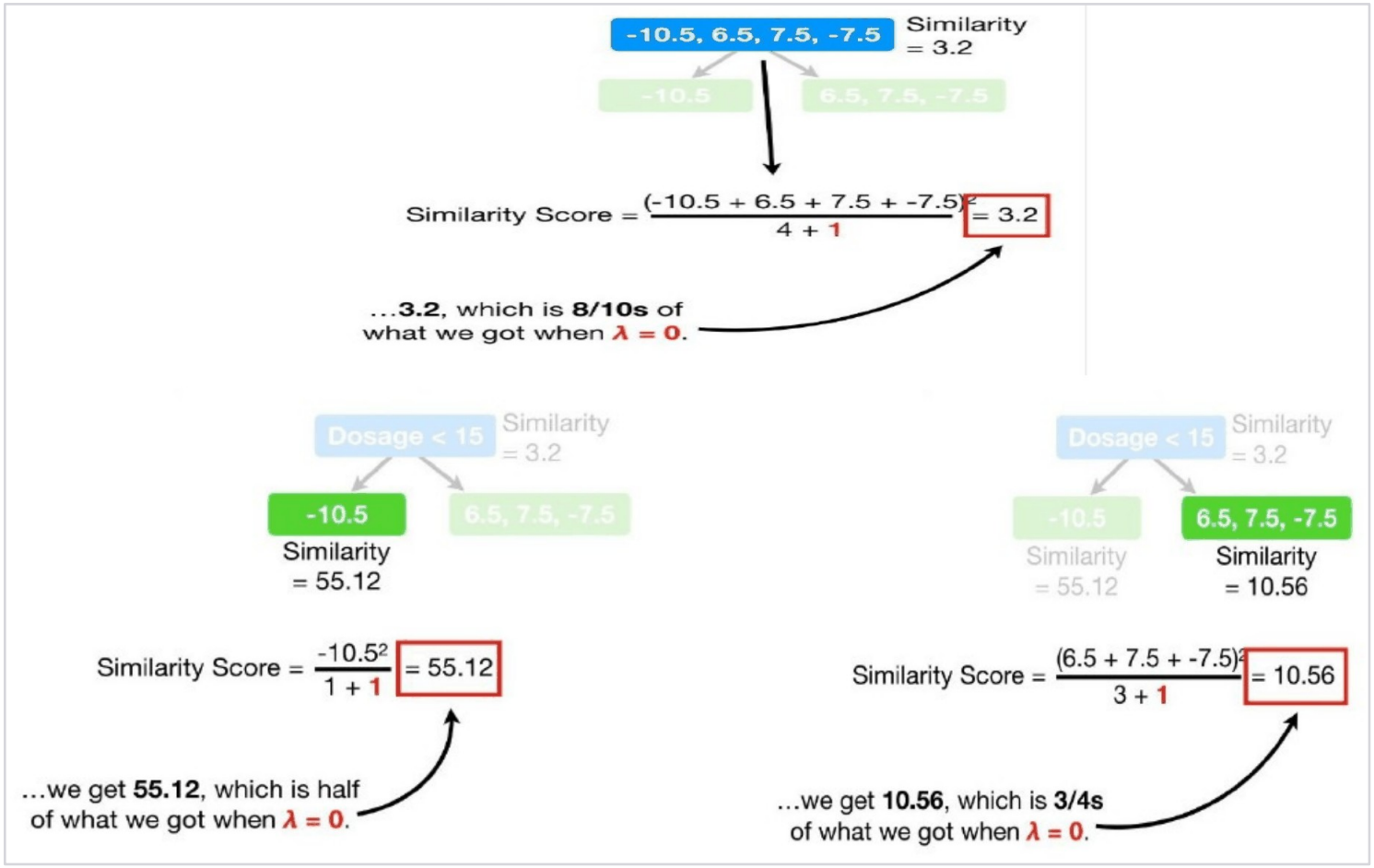
If the lower branch stays, the root stays too, even if its gain (120.33) is below γ.

A larger λ shrinks similarity and makes pruning easier

Similarity and gain recomputed with λ = 1

Effect of λ = 1

- Root similarity falls from 4 to 3.2.
- Leaves with fewer rows shrink more: 110.25 halves to 55.12.
- λ is regularization that lowers prediction sensitivity to individual rows.

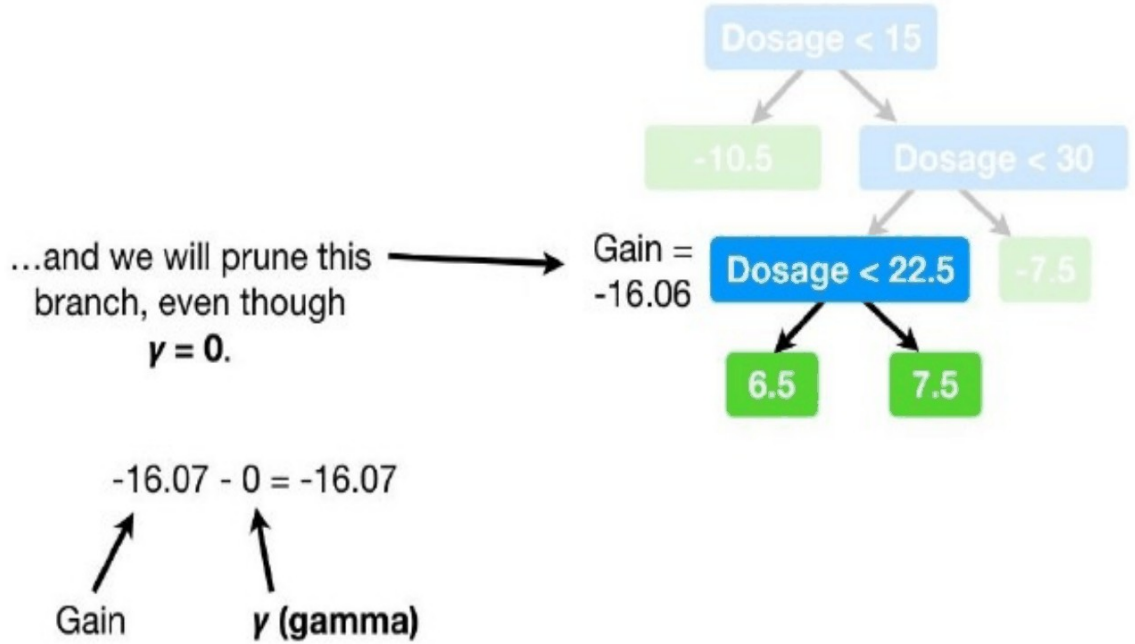


Pruning happens even with γ set to 0

With λ = 1, the gain itself can be negative

How to read it

- At λ = 1, the gain of the Dosage < 22.5 split is −16.06.
- Even at γ = 0, "gain − γ" is negative, so this branch is pruned.
- γ = 0 is not a setting that turns pruning off.



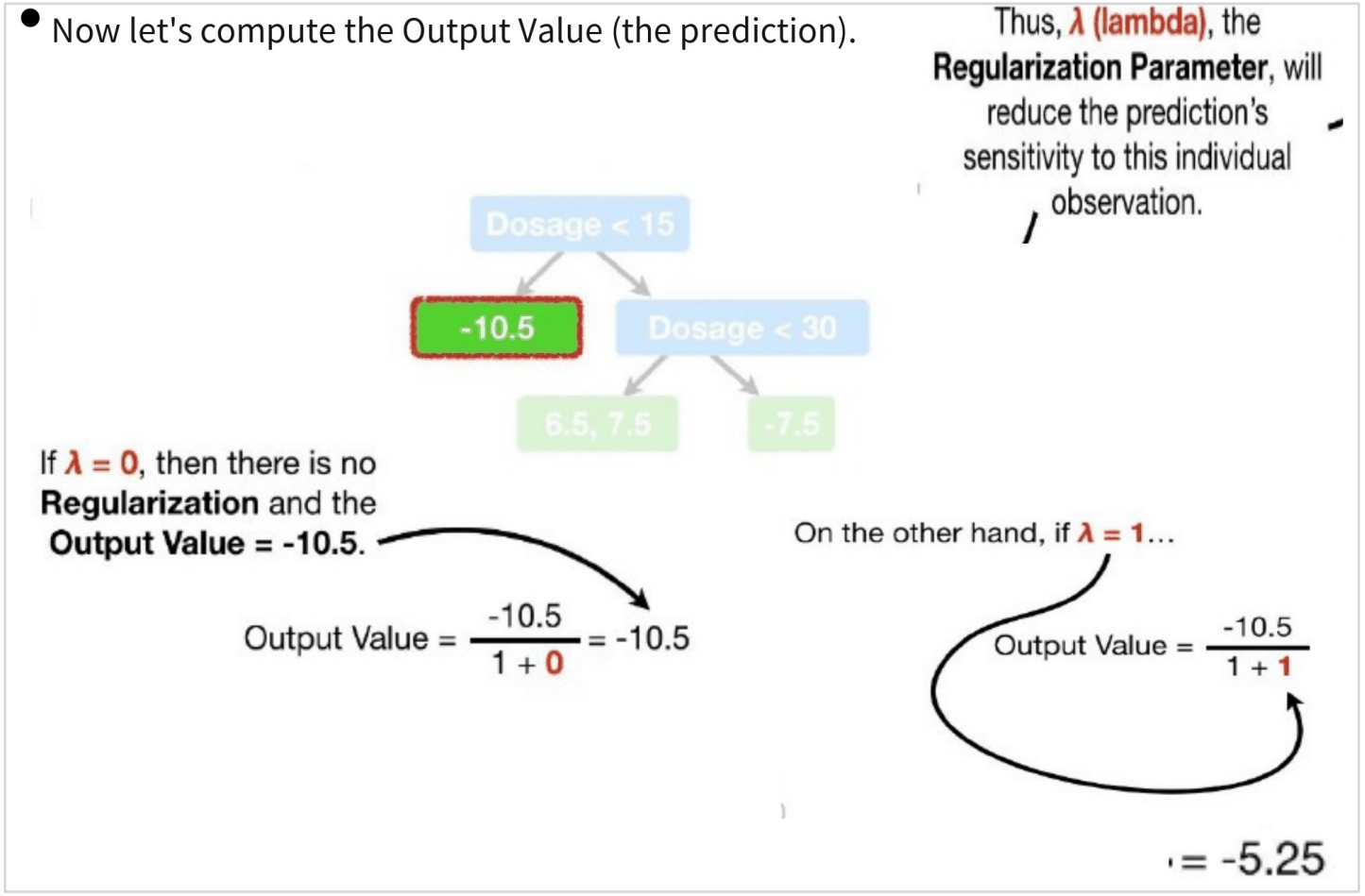
The leaf output is the residual mean shrunk by λ

The output formula is the similarity formula without the square in the numerator

$$\text{Output Value} = \frac{\text{Sum of Residuals}}{\text{Number of Residuals} + \lambda}$$

Change with λ

- At λ = 0, the left leaf output is −10.5.
- At λ = 1, it shrinks to −5.25.
- The output is shrunk so one row cannot sway it.

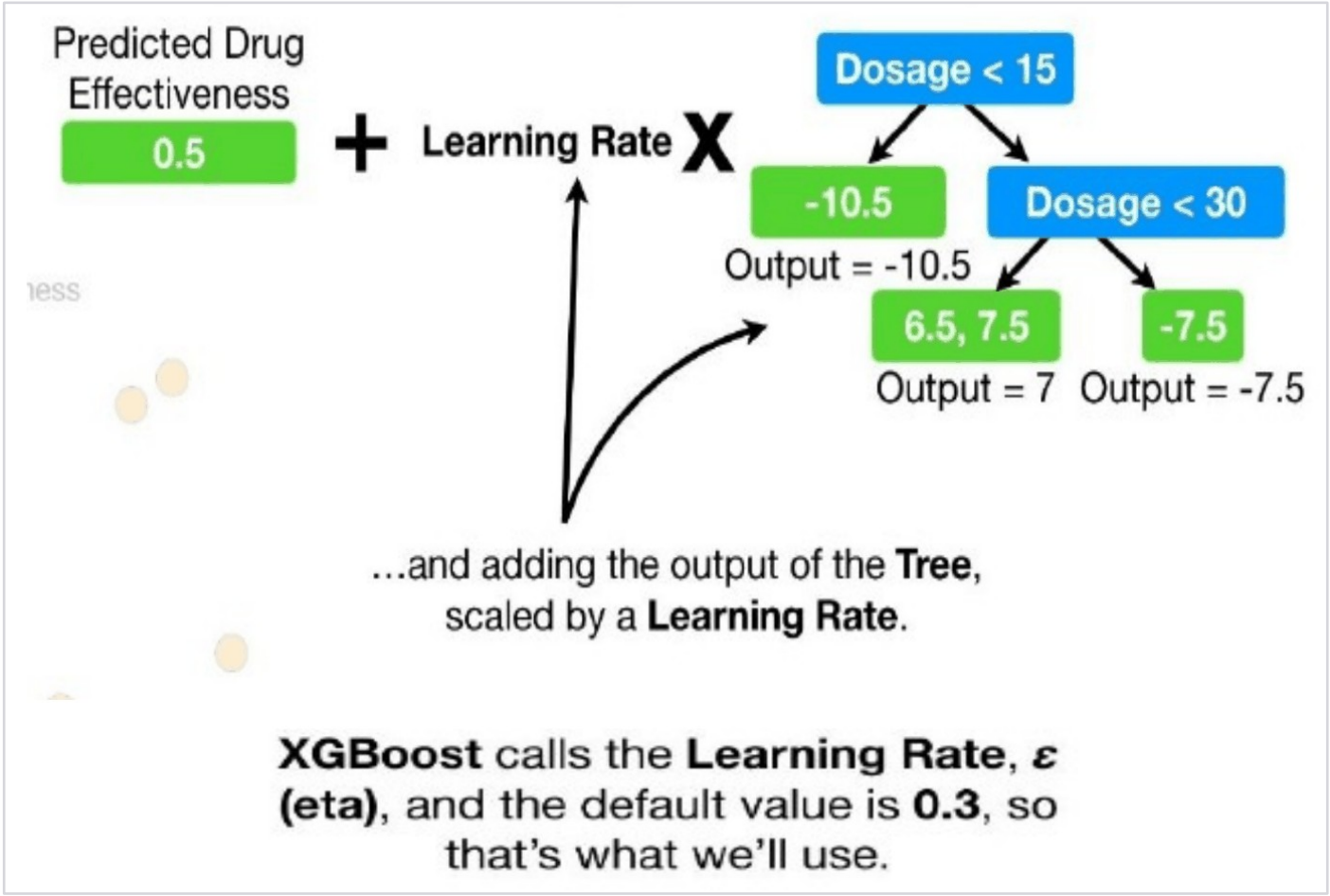


Only a learning rate of 0.3 of the output is applied

The default XGBoost learning rate, eta, is 0.3

How to read it

- The first row's new prediction is 0.5 + 0.3 × (−10.5) = −2.65.
- The residual falls from −10.5 to −7.35, a little closer to the actual value.
- Many small steps prevent overfitting.

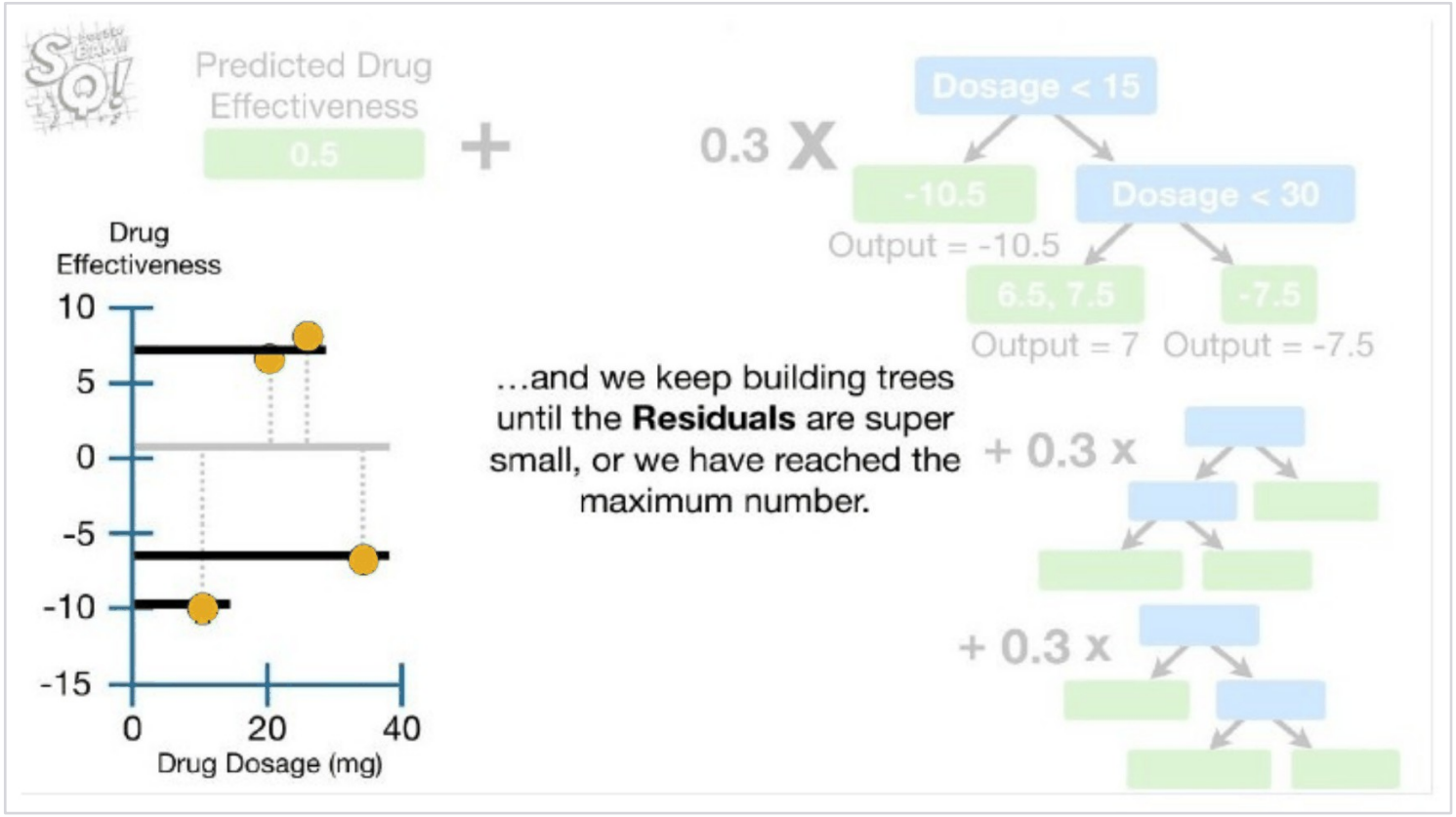


Add trees until the residuals are small enough

Build the next tree from new residuals and add it, scaled by the learning rate

How to read it

- The second tree learns the reduced residuals.
- Stop when residuals are very small or the tree count hits the maximum.
- In practice, stop when validation error no longer falls (early stopping).



One regression tree takes four formulas

Walkthrough recap: group by similarity, choose by gain, prune with γ, set the output value

① Similarity: how alike are the residuals?

$$\text{Similarity} = \frac{(\text{Sum of Residuals})^2}{\text{Number of Residuals} + \lambda}$$

② Gain: how much did the split raise similarity?

$$\text{Gain} = \text{Similarity}_{\text{Left}} + \text{Similarity}_{\text{Right}} - \text{Similarity}_{\text{Root}}$$

③ Pruning: cut if gain is below γ

$$\text{Gain} - \gamma < 0 \quad \Rightarrow \quad \text{Prune the branch}$$

④ Output value: a leaf's prediction adjustment

$$\text{Output Value} = \frac{\text{Sum of Residuals}}{\text{Number of Residuals} + \lambda}$$

XGBoost added six devices for large data

Approximation algorithms and system optimizations on top of the regression tree math

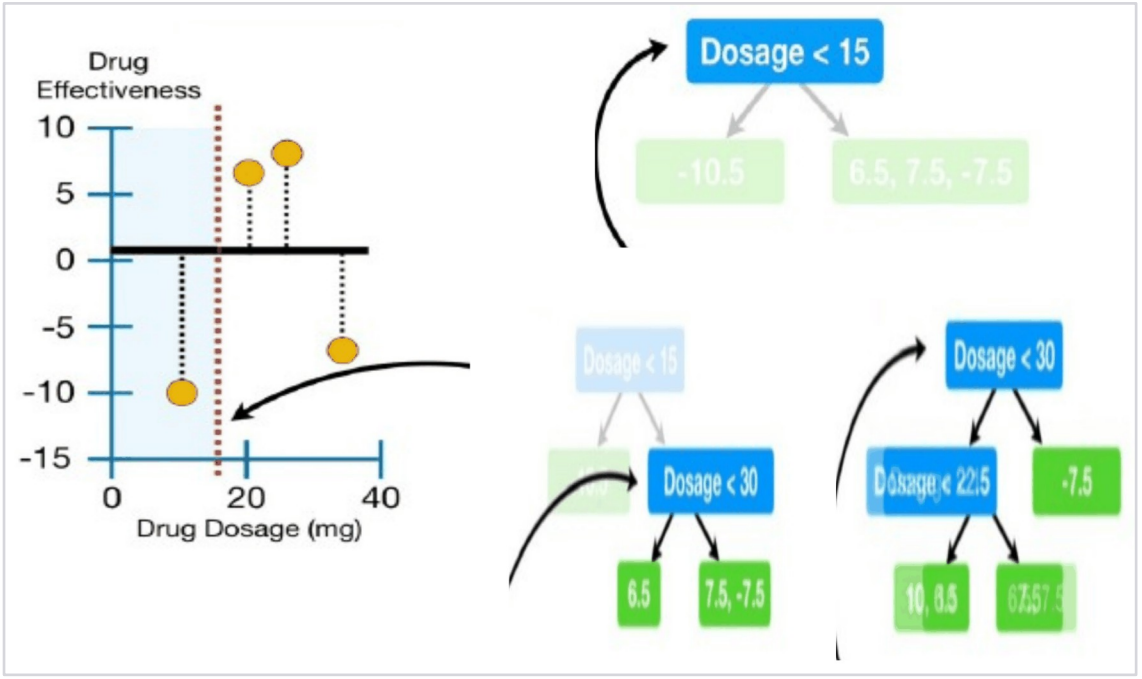
Device	What it does
Approximate greedy algorithm	Tests only quantile boundaries as split candidates, not every value
Parallel learning	Splits the data so several computers calculate at once
Weighted quantile sketch	Forms quantiles with Hessians as weights to build an approximate histogram
Sparsity-aware split finding	Learns the default direction for missing values
Cache-aware access	Keeps gradients in CPU cache for faster computation
Out-of-core blocks	Reads compressed disk blocks piecewise to handle data larger than memory

From the original's "More than Statistics": a good algorithm needs engineering devices to be used in industry.

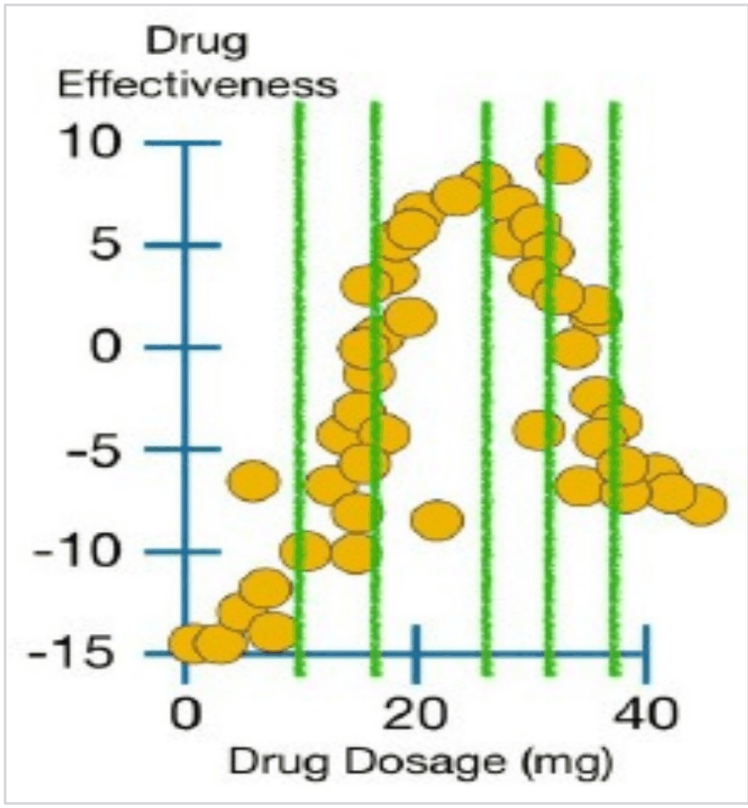
Approximate greedy tests only quantile boundaries

A greedy algorithm that tries every threshold is slow on large data

Greedy algorithm: tests every threshold, one step at a time



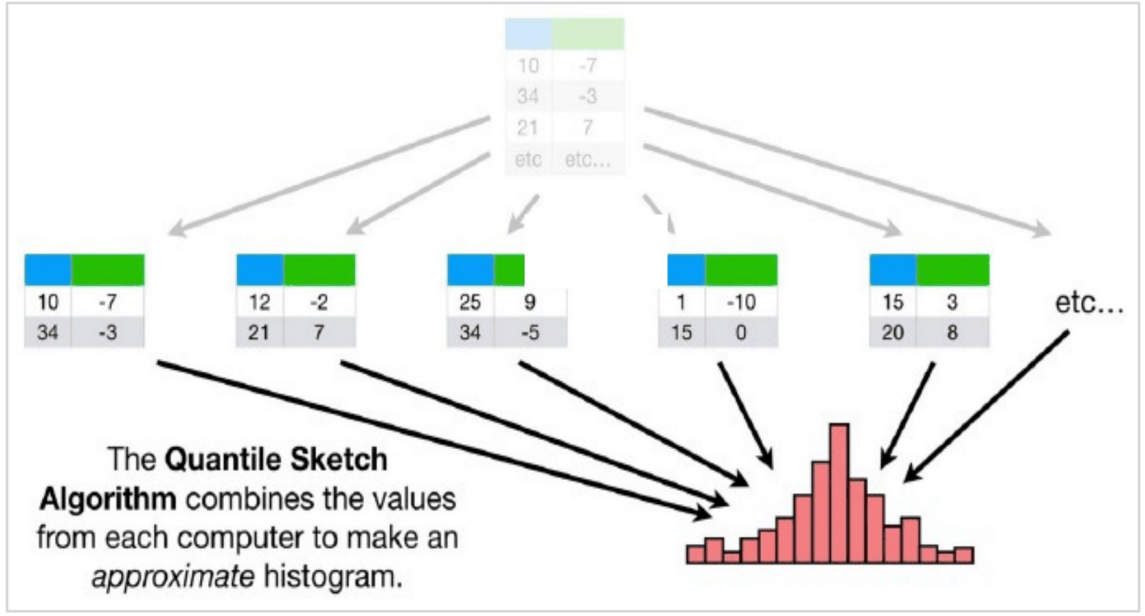
Approximate greedy: tests only quantile boundaries (green lines)



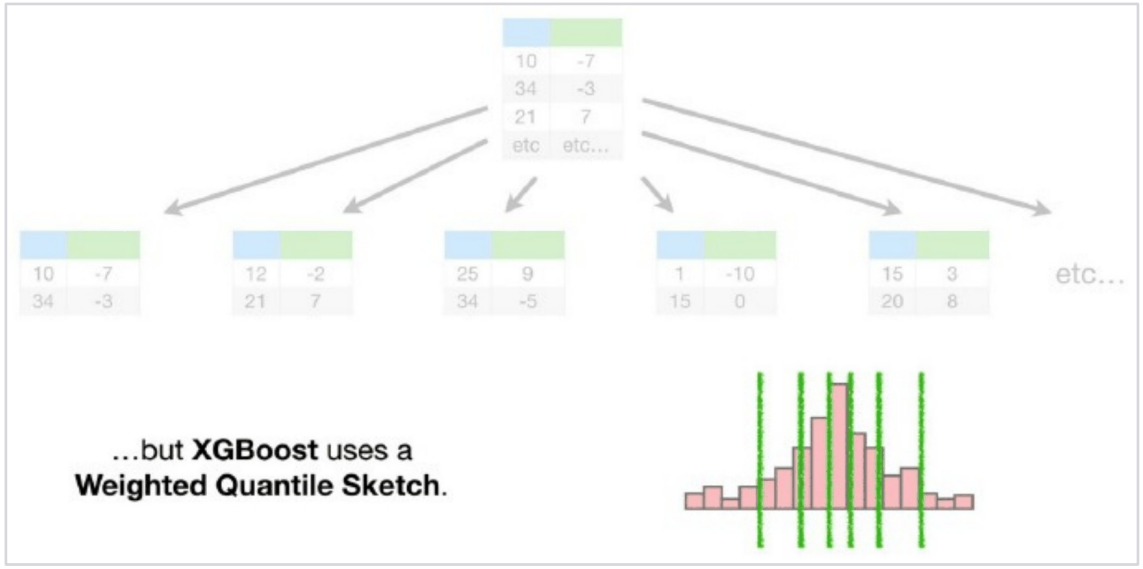
Quantiles are computed on data pieces, then merged

Parallel learning summarizes values per piece and merges them into an approximate histogram

Quantile sketch: each computer's summary merges into an approximate histogram



XGBoost uses a weighted quantile sketch that reflects weights

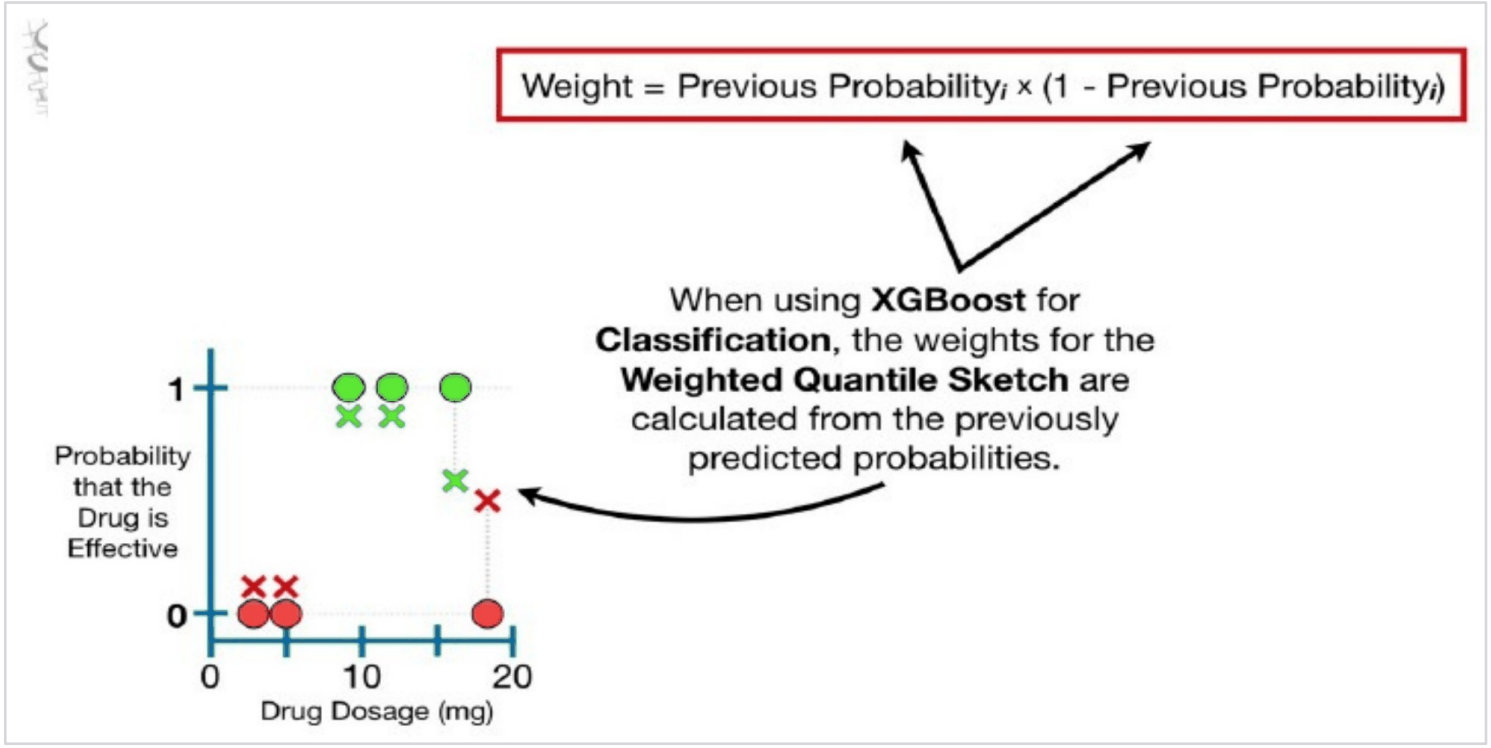


The weight is the uncertainty of the previous prediction

In classification, weight = previous probability × (1 – previous probability)

How to read it

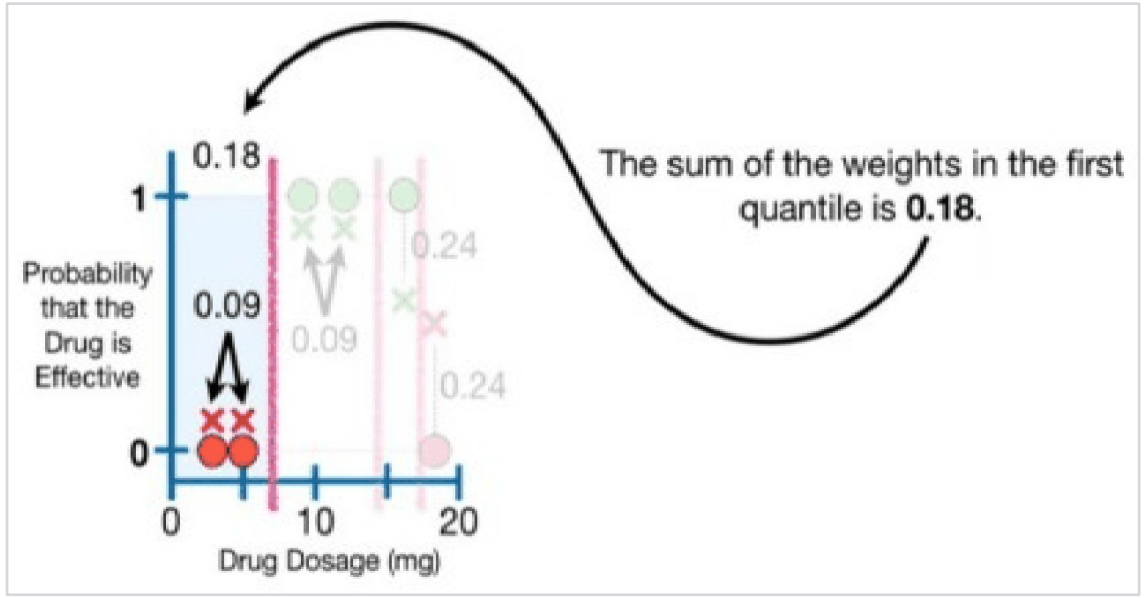
- This weight equals the Hessian of the log loss.
- The closer the probability is to 0.5, the larger the weight.
- Rows already predicted with confidence get small weights.



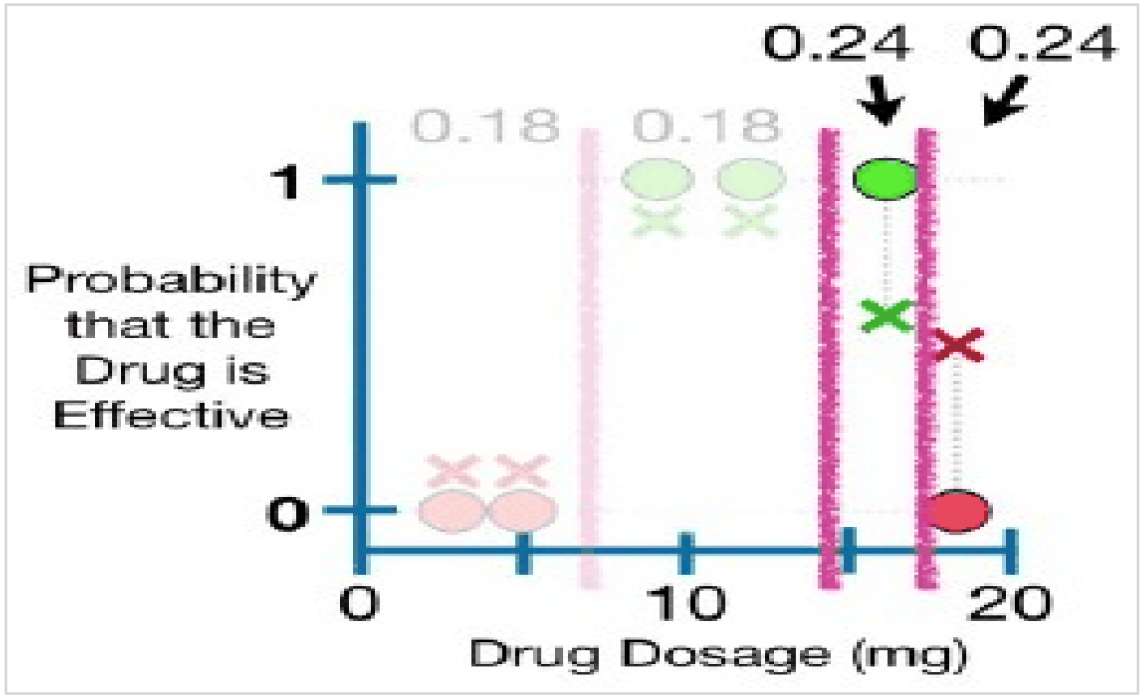
Boundaries are set so each quantile has the same weight sum

Dividing by weight rather than count splits uncertain regions more finely

In the first quantile, several rows sum to a weight of 0.18



The last quantile reaches weight 0.24 with a single row

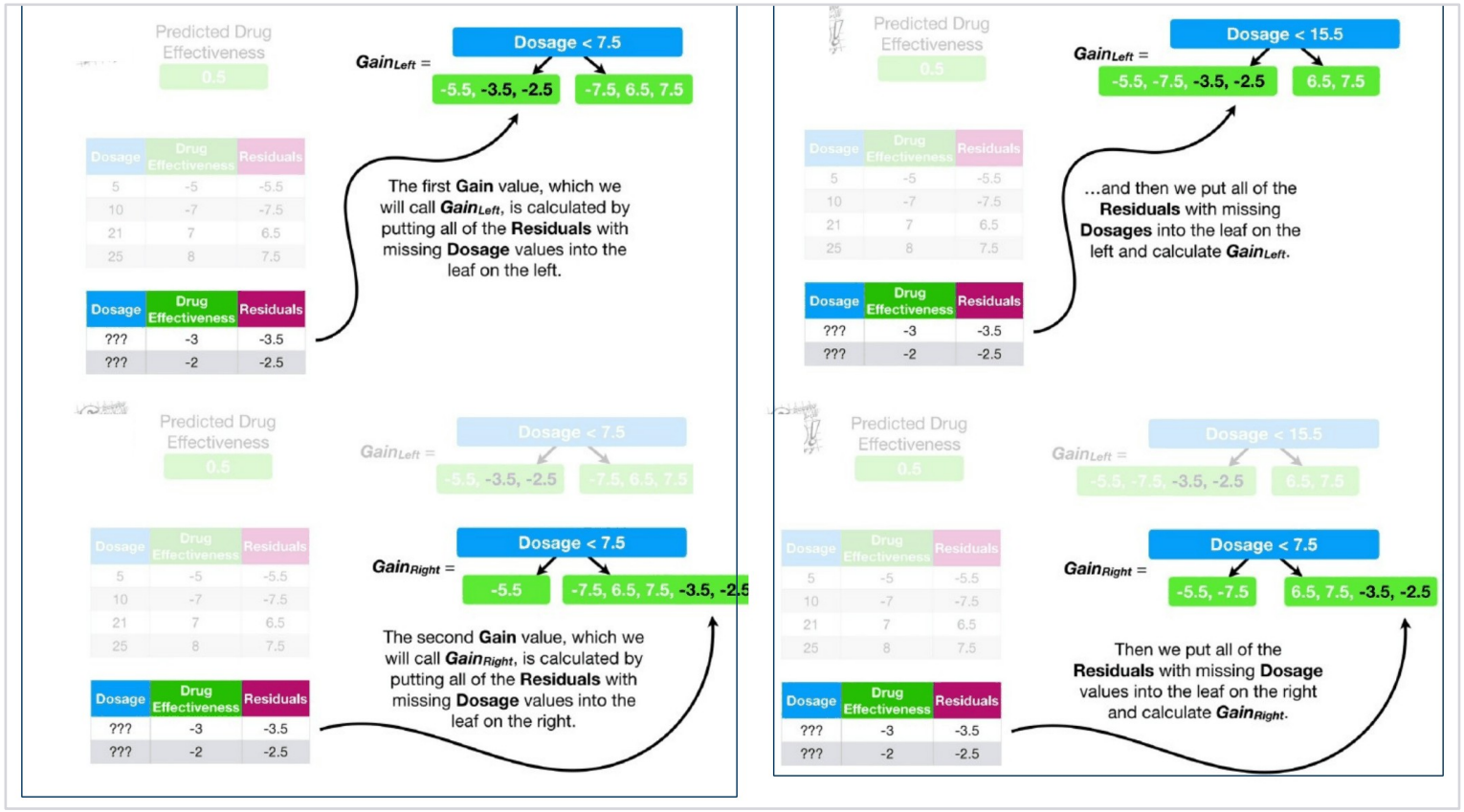


Missing values go to the side with higher gain

Sparsity-aware split finding: try the rows with missing values on both sides

How to read it

- Compare the gain with missing residuals on the left and on the right.
- The larger one sets the default direction of the split.
- The same principle applies to the many zeros from one-hot encoding.

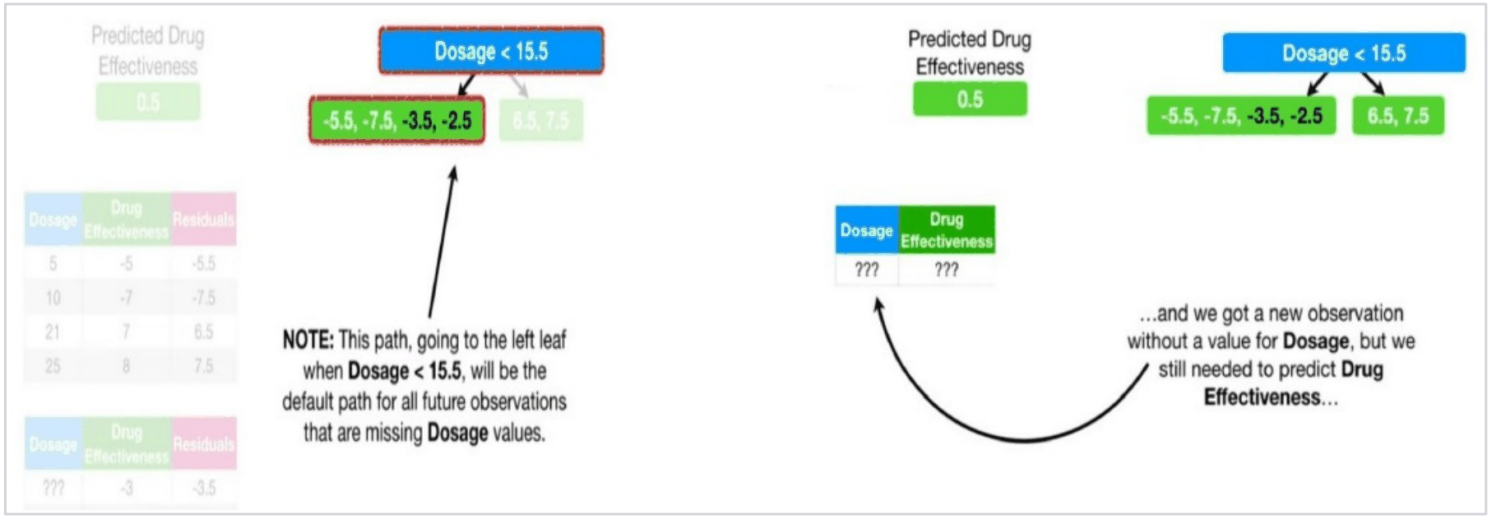


New missing values follow the learned default path

The direction set in training handles missing values at prediction time

How to read it

- Training rows with missing values go to the leaf on the default side.
- A new row with no dosage follows the same path to a prediction.
- Training can start right away, with no imputation of missing values.

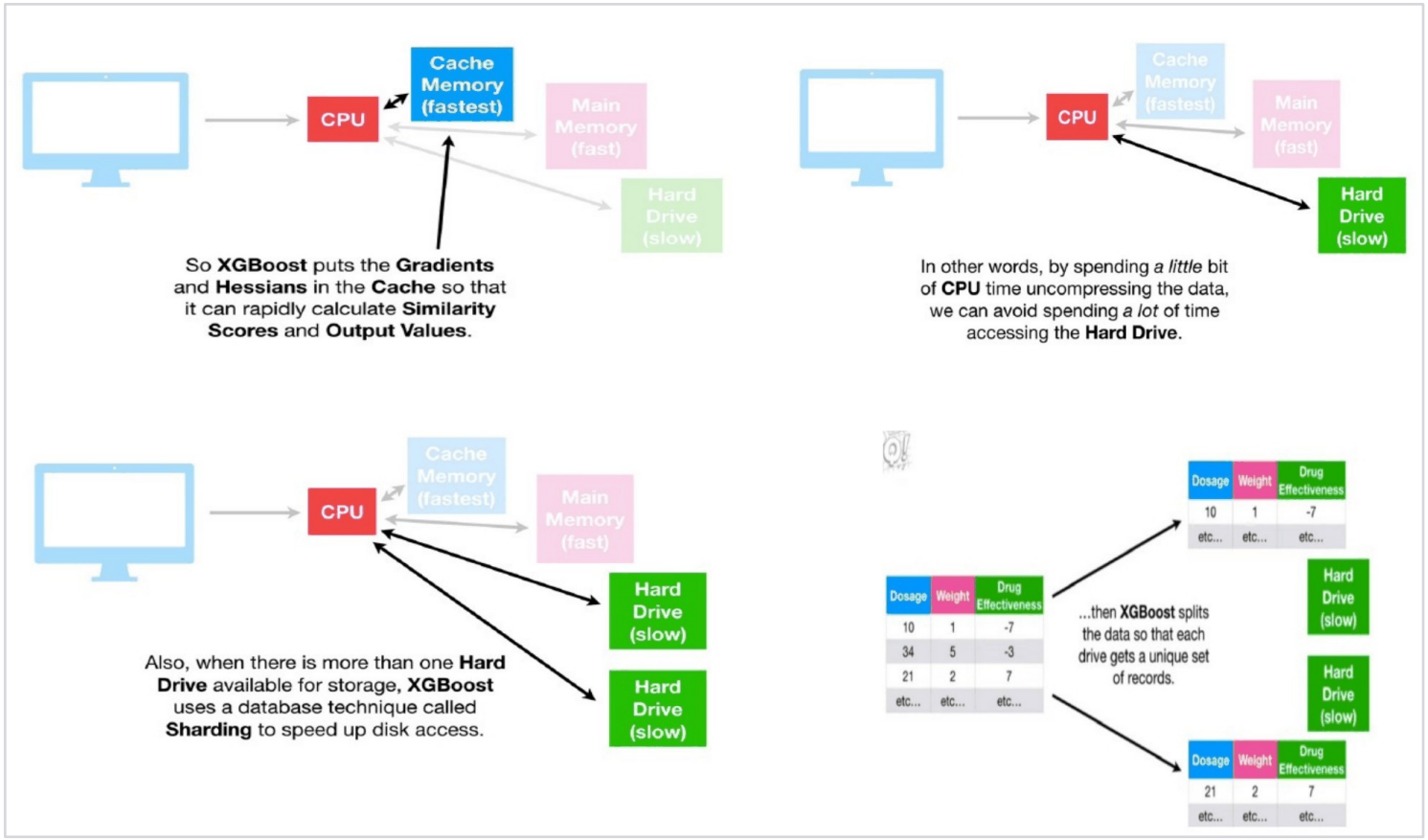


Cache and disk awareness raise computation speed

Cache-aware access and out-of-core computing

How to read it

- Gradients and Hessians sit in CPU cache, so similarity is computed quickly.
- On disk, compressed blocks reduce reads; with several disks, data is split across them.
- Since 3.0, out-of-core training is also faster on GPUs.



Random rows and columns make each tree learn differently

subsample and colsample_bytree: ideas borrowed from random forests

How to read it

- Each tree uses only some of the rows.
- Each tree uses only some of the features.
- Speed rises, correlation between trees falls, and overfitting falls.



02

PART 2 · LightGBM

LightGBM: Boosting That Sees Less and Finds Faster

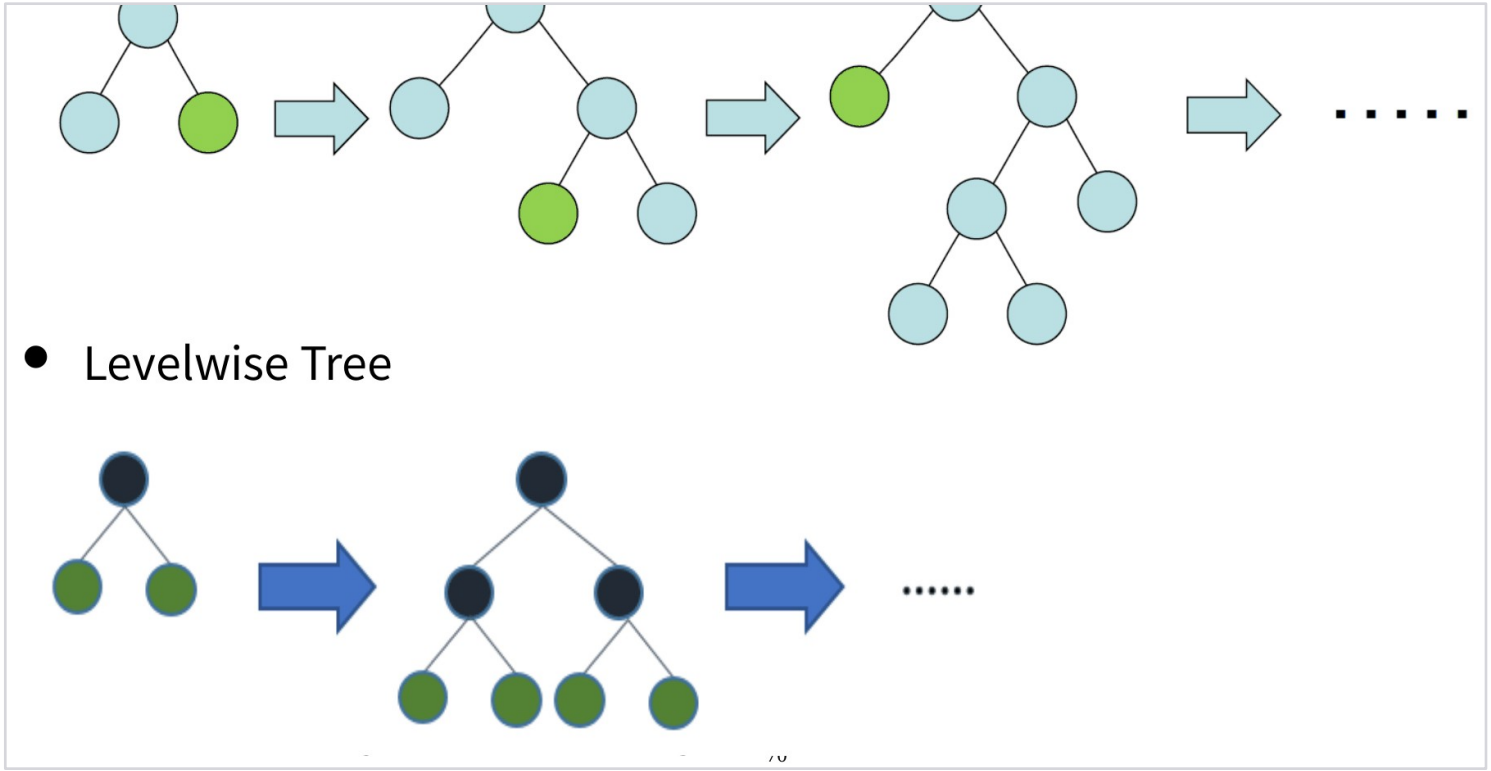
How leaf-wise growth, GOSS and EFB cut time and memory

LightGBM trees dig deep from the highest-loss leaf

Leaf-wise growth: top, LightGBM; bottom, XGBoost's level-wise growth

How to read it

- Only the one leaf with the largest gain is split.
- Less important sides do not grow, so loss falls faster at the same leaf count.
- Trees deepen easily, so num_leaves and max_depth hold them back.

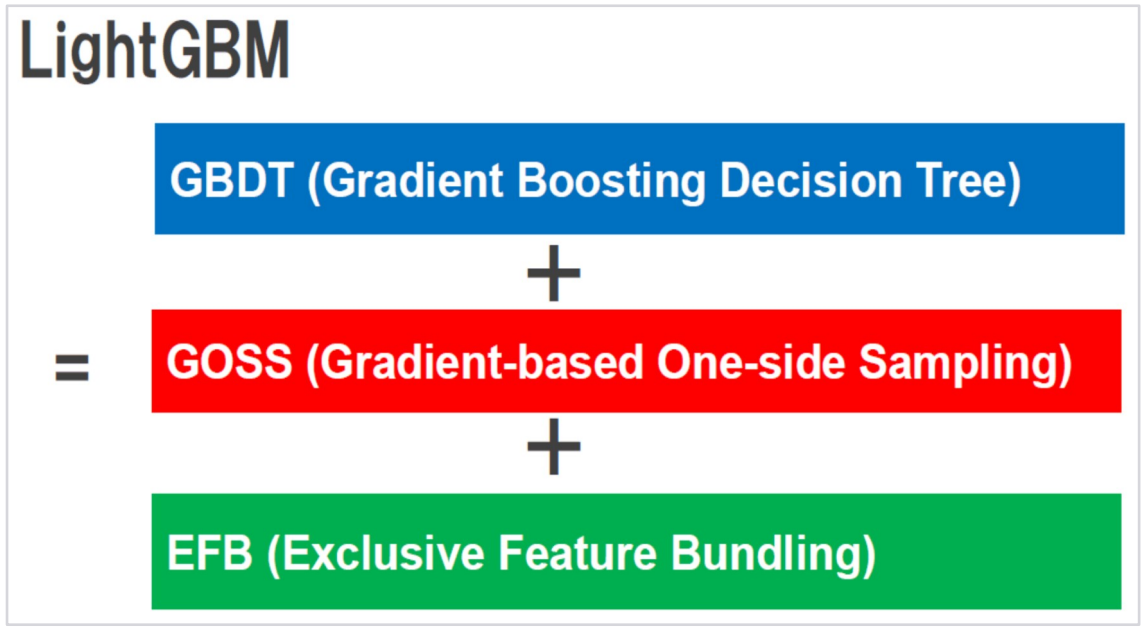


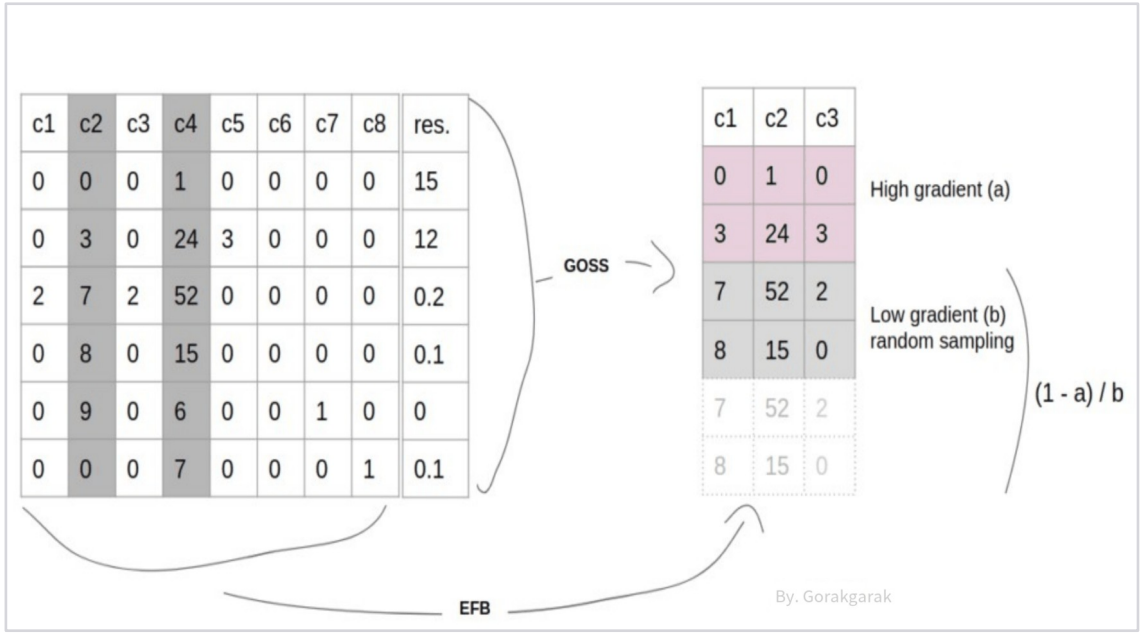
LightGBM added two techniques to GBDT

GOSS reduces the rows to compute; EFB reduces the columns

LightGBM = GBDT + GOSS + EFB

Select rows by gradient; bundle mutually exclusive columns



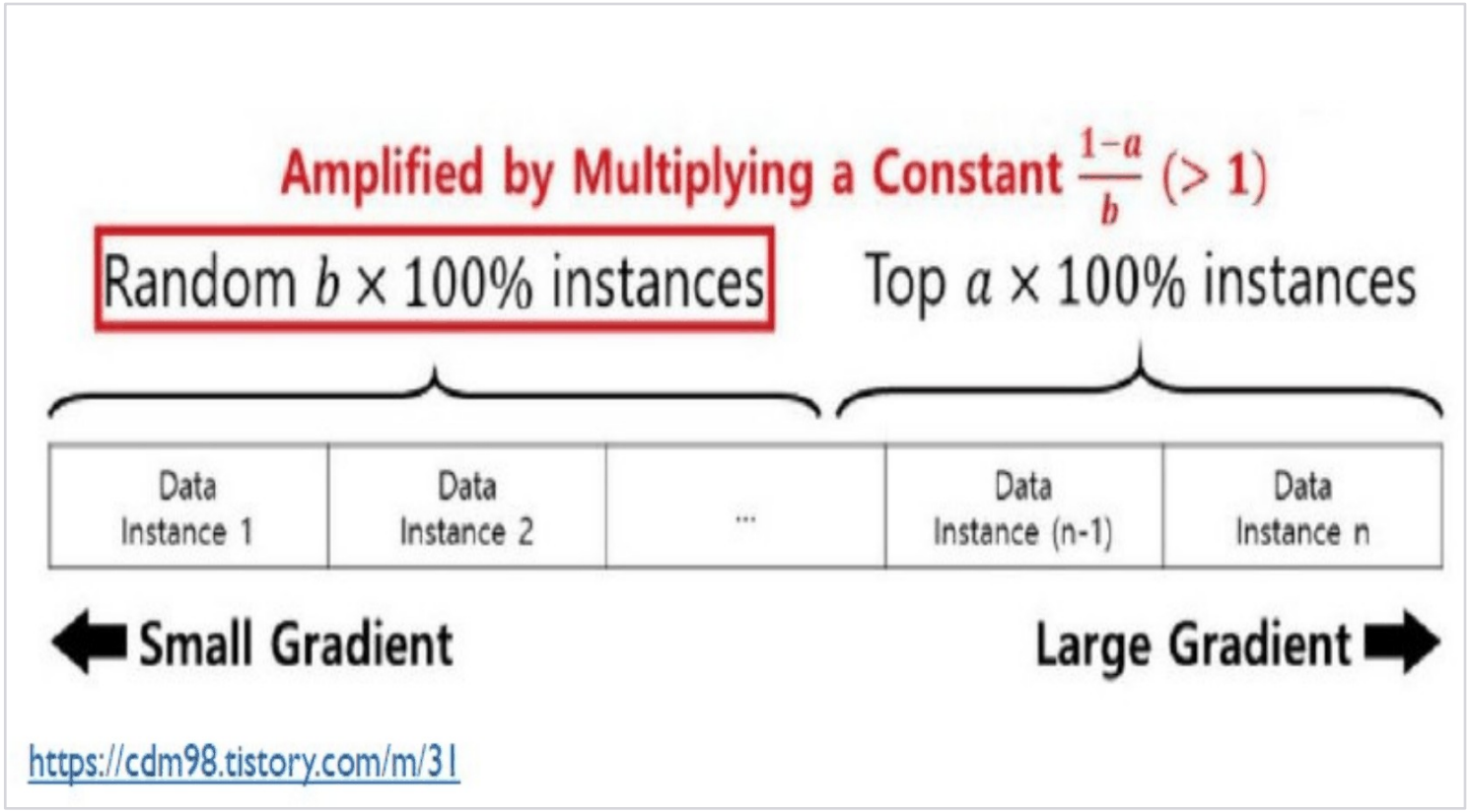


GOSS selects rows by gradient size

Gradient-based One-Side Sampling — keep every large-gradient row, sample only some of the rest

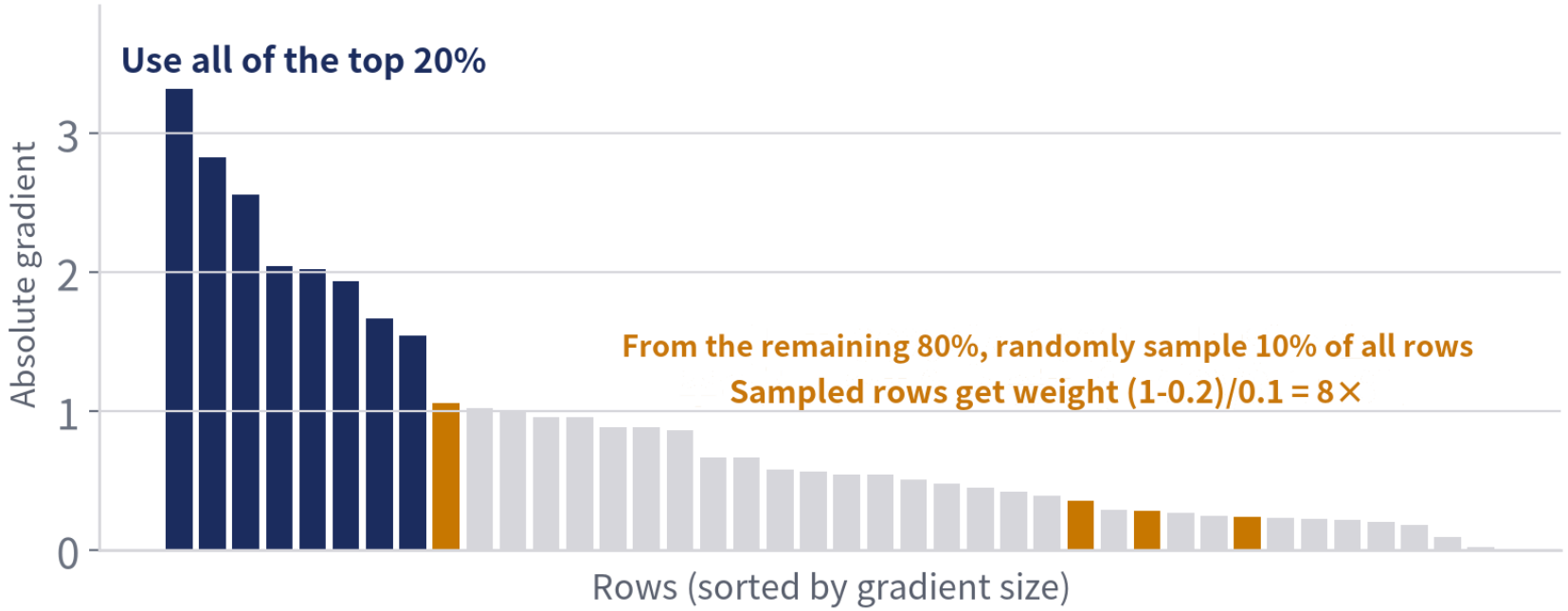
How to read it

- Rows with large gradients are still under-trained.
- Keep the top a share, then randomly sample b of all rows from the rest.
- Multiply the drawn rows by a constant to correct the original distribution.



With a = 0.2 and b = 0.1, 30% of the data finds the split

GOSS sampling example: the small-gradient side is corrected by weighting the sample (※ example data)



$$\frac{1-a}{b}$$

Correction constant

- In the example, 0.8 divided by 0.1 gives 8.
- The small-gradient sum is restored to its original size.

EFB bundles features that are rarely nonzero together

Exclusive Feature Bundling: count how often two features are nonzero together (conflicts)

How to read it

- The left table is sparse data with 10 rows and 5 features.
- The right table counts conflicts for each feature pair.
- Merging features with few conflicts loses little information.

	x_1	x_2	x_3	x_4	x_5
I_1	1	1	0	0	1
I_2	0	0	1	1	1
I_3	1	2	0	0	2
I_4	0	0	2	3	1
I_5	2	1	0	0	3
I_6	3	3	0	0	1
I_7	0	0	3	0	2
I_8	1	2	3	4	3
I_9	1	0	1	0	0
I_{10}	2	3	0	0	2

	x_1	x_2	x_3	x_4	x_5
x_1	-	6	2	1	6
x_2	6	-	1	1	6
x_3	2	1	-	3	4
x_4	1	1	3	-	3
x_5	6	6	4	3	-

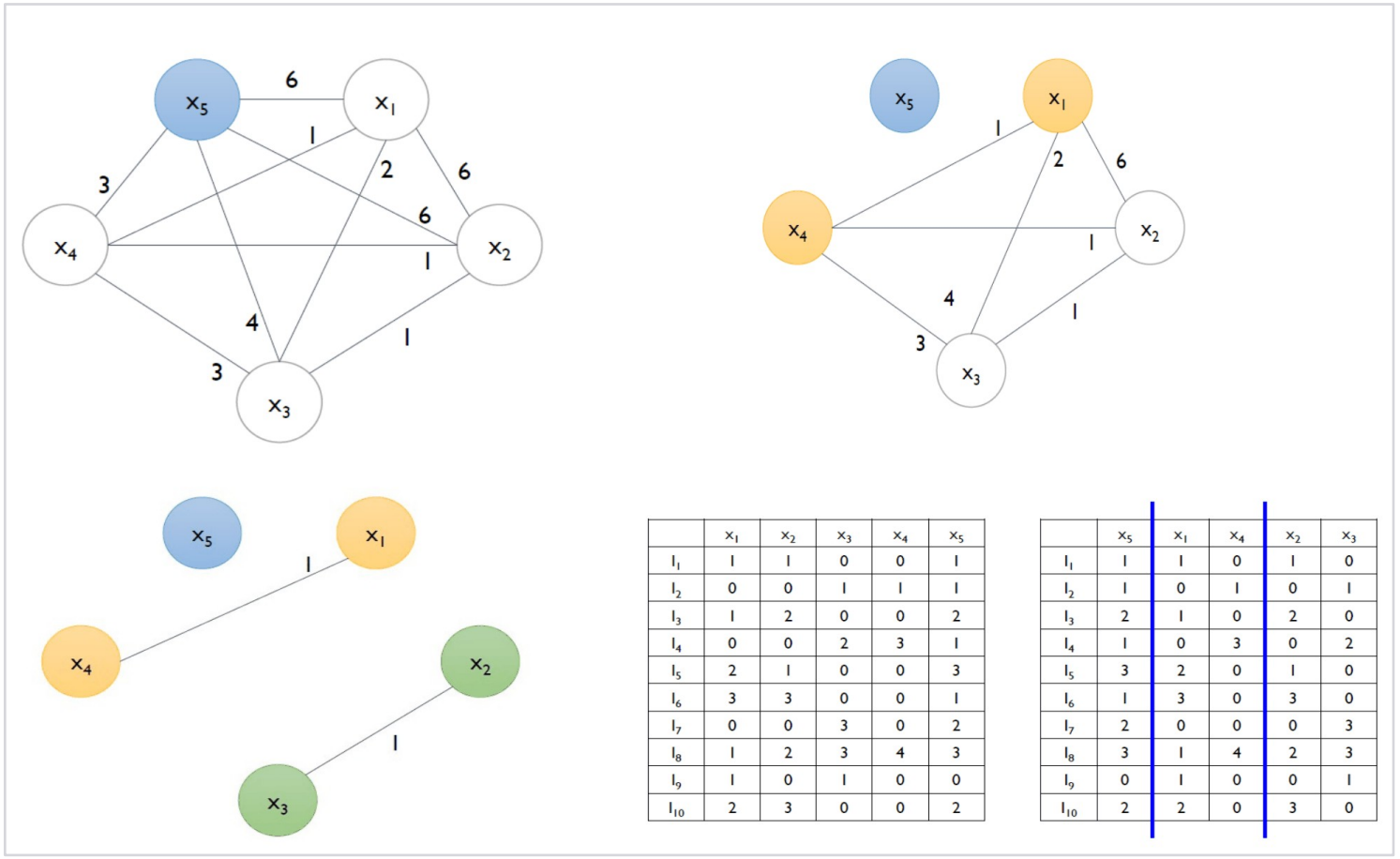
	x_5	x_1	x_2	x_3	x_4
d	19	15	14	10	8

Feature bundles are set as if coloring a conflict graph

Conflict threshold 0.2: features with the most links are assigned to bundles first

How to read it

- Features are vertices; conflict counts are edge weights.
- Features joined by an edge above the threshold go to different bundles.
- The 5 features shrink to fewer bundles.



	x_1	x_2	x_3	x_4	x_5
I_1	1	1	0	0	1
I_2	0	0	1	1	1
I_3	1	2	0	0	2
I_4	0	0	2	3	1
I_5	2	1	0	0	3
I_6	3	3	0	0	1
I_7	0	0	3	0	2
I_8	1	2	3	4	3
I_9	1	0	1	0	0
I_{10}	2	3	0	0	2

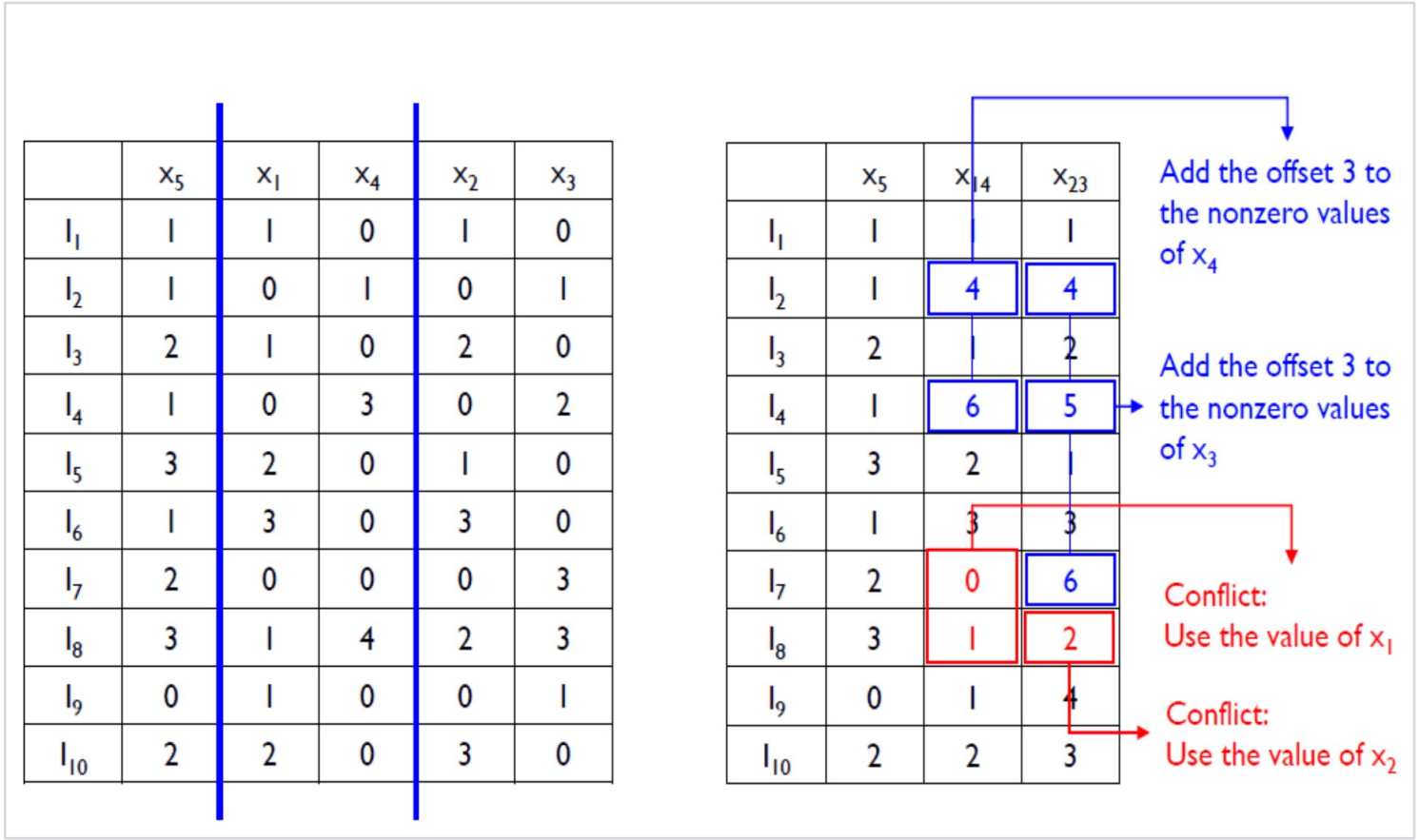
	x_5	x_1	x_4	x_2	x_3
I_1	1	1	0	1	0
I_2	1	0	1	0	1
I_3	2	1	0	2	0
I_4	1	0	3	0	2
I_5	3	2	0	1	0
I_6	1	3	0	3	0
I_7	2	0	0	0	3
I_8	3	1	4	2	3
I_9	0	1	0	0	1
I_{10}	2	2	0	3	0

Bundling adds an offset to keep the original features distinct

Add the base feature's maximum so the value ranges do not overlap

How to read it

- Add 3 to the nonzero values of x4 to separate its range from x1.
- If both features are nonzero, that is a conflict, and one value is chosen.
- One histogram on the bundled feature reduces computation.



	x_5	x_1	x_4	x_2	x_3
I_1	1	1	0	1	0
I_2	1	0	1	0	1
I_3	2	1	0	2	0
I_4	1	0	3	0	2
I_5	3	2	0	1	0
I_6	1	3	0	3	0
I_7	2	0	0	0	3
I_8	3	1	4	2	3
I_9	0	1	0	0	1
I_{10}	2	2	0	3	0

	x_5	x_{14}	x_{23}
I_1	1	1	1
I_2	1	4	4
I_3	2	1	2
I_4	1	6	5
I_5	3	2	1
I_6	1	3	3
I_7	2	0	6
I_8	3	1	2
I_9	0	1	4
I_{10}	2	2	3

LightGBM tuning starts with num_leaves

Key parameters against leaf-wise overfitting (per LightGBM 4.7 docs)

Parameter	Default	How to adjust
num_leaves	31	Most important. Set below the leaf count max_depth could produce
max_depth	−1 (no limit)	Limit it too when trees grow too deep
min_data_in_leaf	20	Raising it removes small leaves and reduces overfitting
max_bin	255	Lowering it is faster but coarsens split candidates
feature_fraction	1.0	Lower to 0.7–0.9 so each tree uses different features

Since 4.7.0 (July 2026), polars input and NVIDIA multi-GPU training are supported.

03

PART 3 · CatBoost

CatBoost: Boosting That Blocks Leakage with Order

Target encoding leakage, Ordered Boosting and symmetric trees

Learning residuals on the same data biases predictions

The classic boosting procedure and the problem CatBoost points out

Classic boosting procedure

- Residuals are the differences between actual values and their mean.
- Build a model that learns the residuals.
- Add predictions scaled by the learning rate, and repeat.

Problems

- Training is slow.
- Computing residuals and retraining on the same rows causes overfitting.
- The paper calls this prediction shift.

CatBoost applies an order principle, "use only earlier rows," to both encoding and residuals.

Target encoding replaces a category with the target mean

Target (Mean) Encoding example: feature 1 predicts the day's high temperature (class_label)

time	feature 1	class_label
sunday	sunny	35
monday	sunny	32
tues	cloudy	15
wed	cloudy	14
thurs	mostly_cloudy	10
fri	cloudy	20
sat	cloudy	25

Calculation and problem

- Replace cloudy with its class_label mean: (15 + 14 + 20 + 25) / 4 = 18.5.
- Columns do not grow, however many categories there are.
- But the target being predicted enters the input, causing data leakage.

A row's own target lets encoding reveal the answer

StatQuest example: the encoded Color < 0.42 alone perfectly separates Troll 2 fans

How to read it

- In a category with few rows, the encoded value is the row's own target.
- The tree uses this one value to split the training data perfectly.
- It fails on new data: classic overfitting.
- Leave-One-Out, averaging without the current row, still has problems.



	Favorite Color	Height (m)	Loves Troll 2
Subset A	0.33	1.77	1
Subset B	0.5	1.32	0
Subset C	0.33	1.81	1
Subset D	0.5	1.56	0
Subset E	0.33	1.64	1
Subset F	0.5	1.61	0
Subset G	0.5	1.73	0

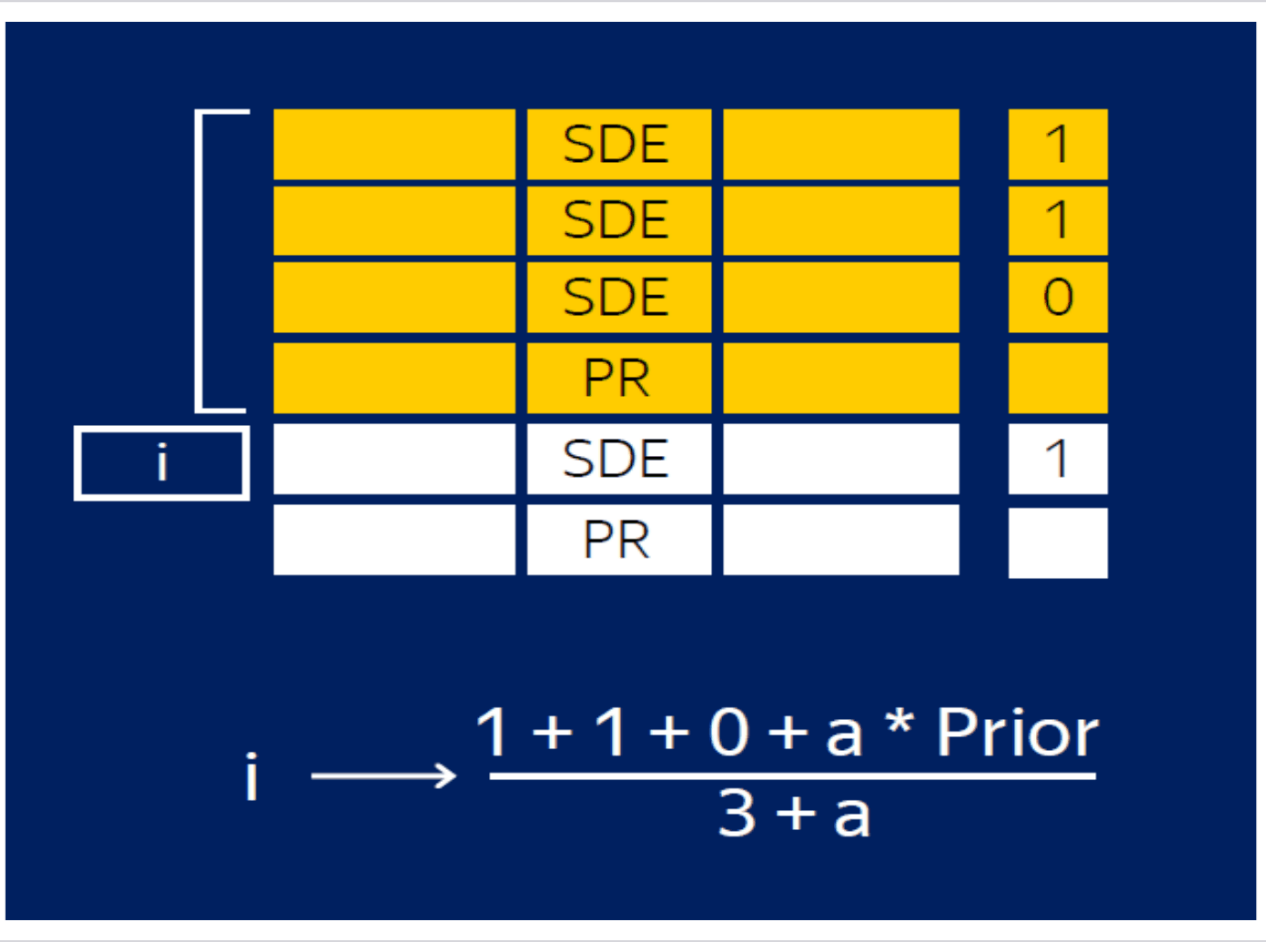
Favorite Color	Height (m)	Loves Troll 2
0	1.77	1
0	1.32	0
0	1.81	1
0	1.56	0
0	1.64	1
0	1.61	0
0	1.73	0

Ordered TS encodes with the targets of earlier rows only

Target Based Statistics: a Prior and weight a stabilize the mean

How to read it

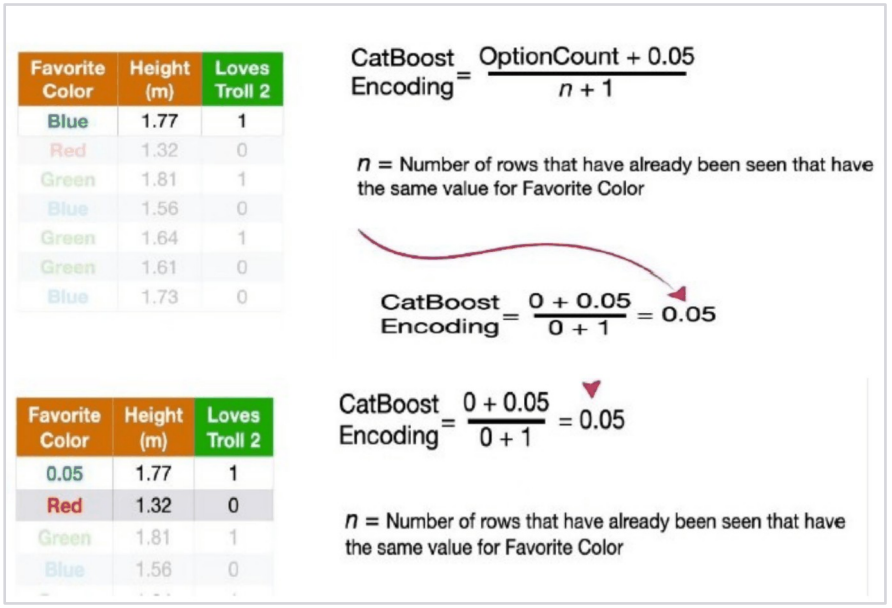
- Row i looks only at same-category rows before it.
- Blend the Prior with SDE's earlier targets 1, 1, 0 and take the mean.
- In the earlier example, Friday's cloudy becomes (15 + 14) / 2 = 14.5 and Saturday's 16.3.

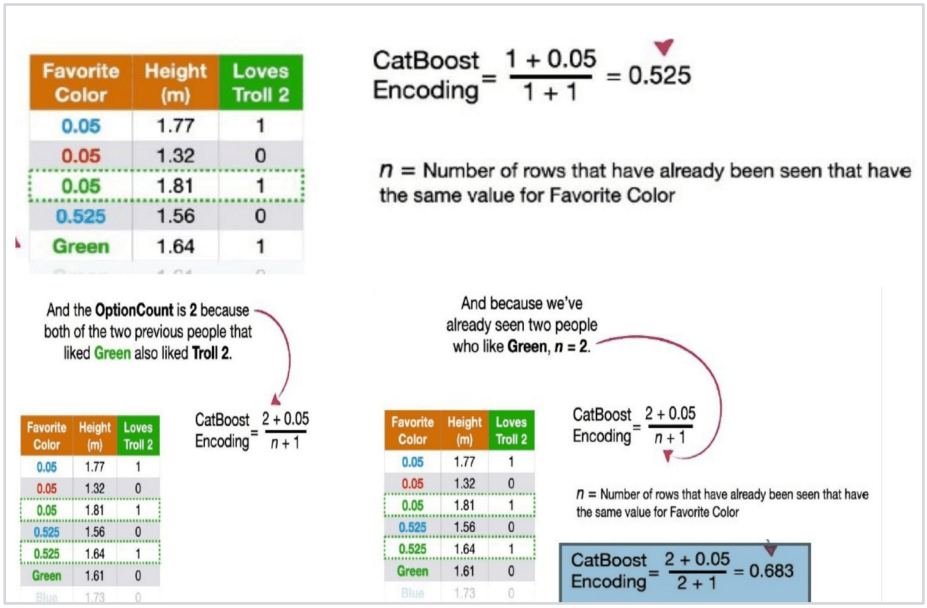


The StatQuest example uses 0.05 as the prior

CatBoost encoding formula: count only same-category rows seen earlier

$$\text{CatBoost Encoding} = \frac{\overbrace{\text{OptionCount}}^{\text{positives seen earlier in the same category}} + 0.05}{\underbrace{n}_{\text{rows seen earlier in the same category}} + 1}$$





The same category gets different values depending on order

Hence the name Ordered Target Encoding

How to read it

- The first Blue is 0.05; the next Blue reflects one earlier Blue's target and is 0.525.
- A row's own target is never used, so nothing leaks.
- CatBoost randomly shuffles rows to average out order effects.

Favorite Color	Height (m)	Loves Troll 2
0.05	1.77	1
0.05	1.32	0
0.05	1.81	1
0.525	1.56	0
0.525	1.64	1
0.683	1.61	0
0.35	1.73	0

$$\text{CatBoost Encoding} = \frac{1 + 0.05}{2 + 1} = 0.35$$

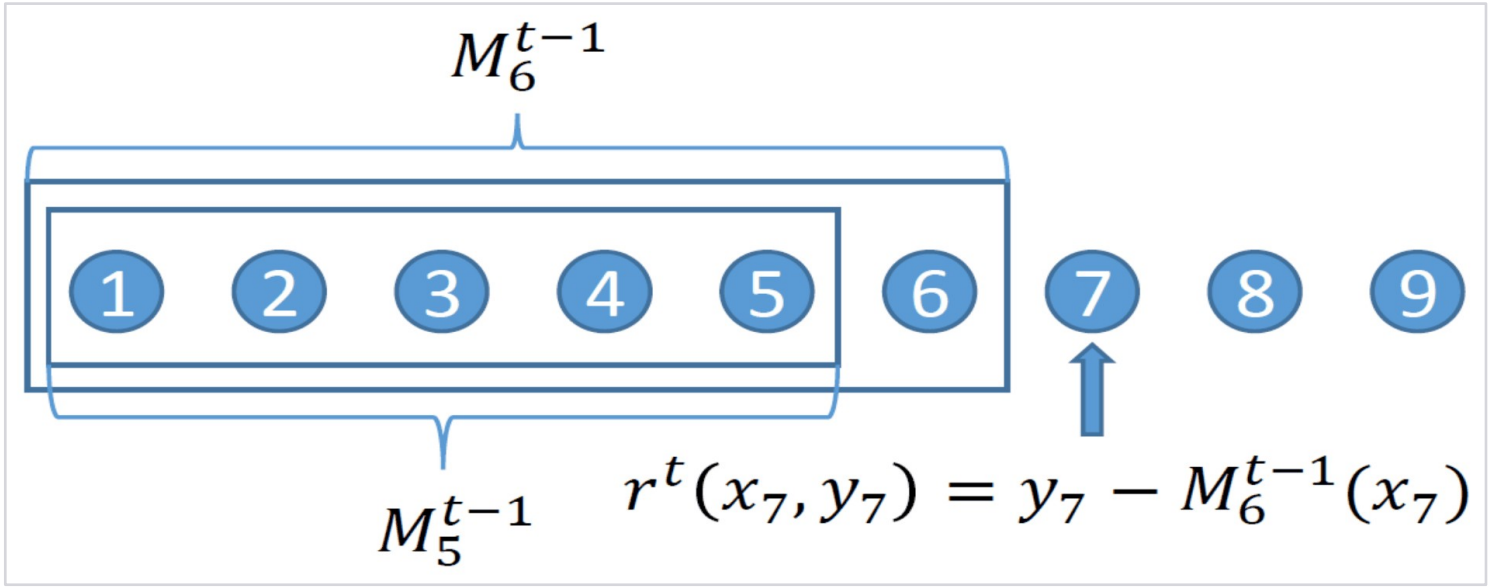
n = Number of rows that have already been seen that have the same value for Favorite Color

Residuals also come only from models of earlier rows

Ordered Boosting: the 7th residual is computed by a model built from rows up to the 6th

How to read it

- A model built on x1's residual predicts x2's residual.
- Models on x1, x2 then predict x3, x4, and so on, growing step by step.
- No residual comes from a model that saw that row, which prevents prediction shift.

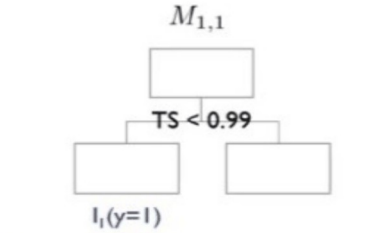


The actual implementation accumulates gradients in order

There is one tree structure; each row's leaf value uses only earlier rows

The first row starts with gradient and leaf value 0

	...	x^j	...	TS	y	G	Δ
I_1	...	A	...	0.000	I	0	0
I_2	...	B	...	1.000	I		
I_3	...	C	...	1.000	I		
I_4	...	A	...	1.000	0		
I_5	...	B	...	0.977	I		
I_6	...	C	...	0.982	I		
I_7	...	B	...	0.992	0		
I_8	...	C	...	0.986	I		
I_9	...	C	...	0.992	0		
I_{10}	...	C	...	0.748	I		



$G(I_1) = 0$

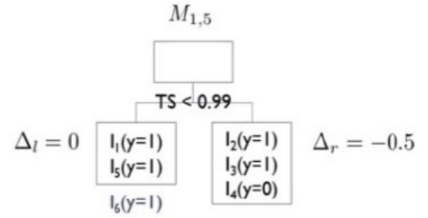
$\Delta(I_1) = 0$

Lecturer's guess

- Initialize both the first gradient and leaf value to 0

The fifth row's leaf value is the mean of earlier rows in the same leaf

	...	x^j	...	TS	y	G	Δ
I_1	...	A	...	0.000	I	0	0
I_2	...	B	...	1.000	I	0	0
I_3	...	C	...	1.000	I	-I	0
I_4	...	A	...	1.000	0	0	-0.5
I_5	...	B	...	0.977	I	-I	0
I_6	...	C	...	0.982	I	-I	-0.5
I_7	...	B	...	0.992	0		
I_8	...	C	...	0.986	I		
I_9	...	C	...	0.992	0		
I_{10}	...	C	...	0.748	I		



$$G(I_6) = grad_{1,\sigma_1(6)-1}(I_6)$$
$$= \Delta_l - y_6 = 0 - 1 = -1$$

$$\Delta(I_6) = \frac{1}{2}(grad_{1,\sigma_1(6)-1}(I_1) + grad_{1,\sigma_1(6)-1}(I_5))$$
$$= \frac{1}{2}(0 - 1) = -0.5$$

After all rows, cosine similarity evaluates the split

Check whether the ordered leaf values point the same way as the actual gradients

How to read it

- The cosine similarity of the two vectors is the candidate split's score.
- Compute this score for each candidate and choose the best split.
- For the example candidate TS < 0.99, the value is 0.3575.

	...	x^j	...	TS	y	G	Δ
I_1	...	A	...	0.000	1	0	0
I_2	...	B	...	1.000	1	0	0
I_3	...	C	...	1.000	1	-1	0
I_4	...	A	...	1.000	0	0	-0.5
I_5	...	B	...	0.977	1	-1	0
I_6	...	C	...	0.982	1	-1	-0.5
I_7	...	B	...	0.992	0	-0.5	-0.33
I_8	...	C	...	0.986	1	-1.5	-0.67
I_9	...	C	...	0.992	0	-0.33	-0.38
I_{10}	...	C	...	0.748	1	-1.67	-0.58

foreach *step of top-down procedure* **do**
  foreach *candidate split* c **do**
    $T_c \leftarrow$ add split c to T;
    if $Mode = Plain$ **then**
      $\Delta(i) \leftarrow \text{avg}(grad_r(p)$ for $p : leaf_r(p) = leaf_r(i))$ for $i = 1..n$;
    if $Mode = Ordered$ **then**
      $\Delta(i) \leftarrow \text{avg}(grad_{r,\sigma_r(i)-1}(p)$ for $p : leaf_r(p) = leaf_r(i), \sigma_r(p) < \sigma_r(i))$ for $i = 1..n$;
    $loss(T_c) \leftarrow \cos(\Delta, G)$
  $T \leftarrow \arg\min_{T_c}(loss(T_c))$

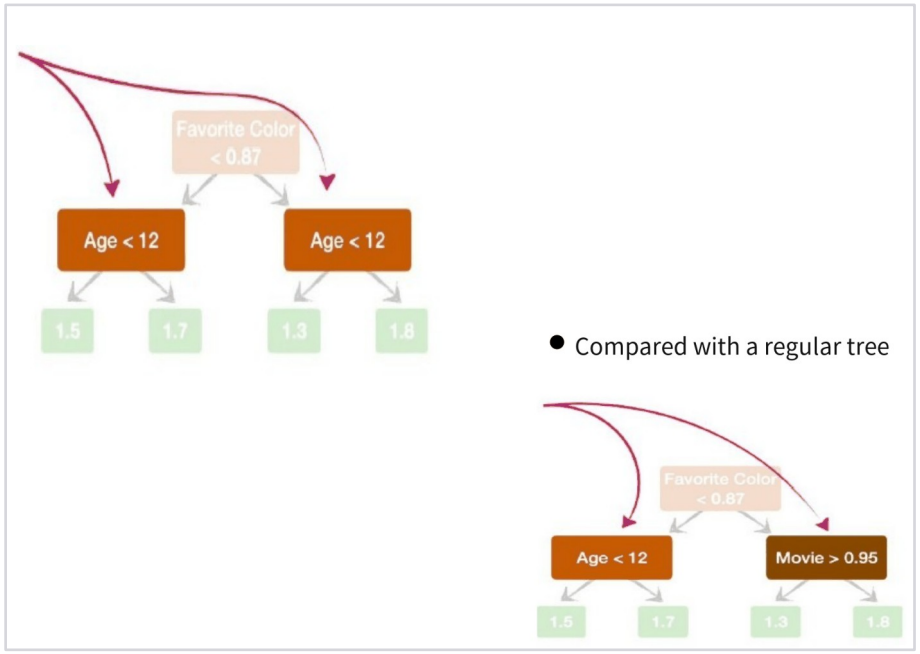
$$loss(T_c) = cos(\Delta, G) = 0.3575$$

(TS < 0.99)

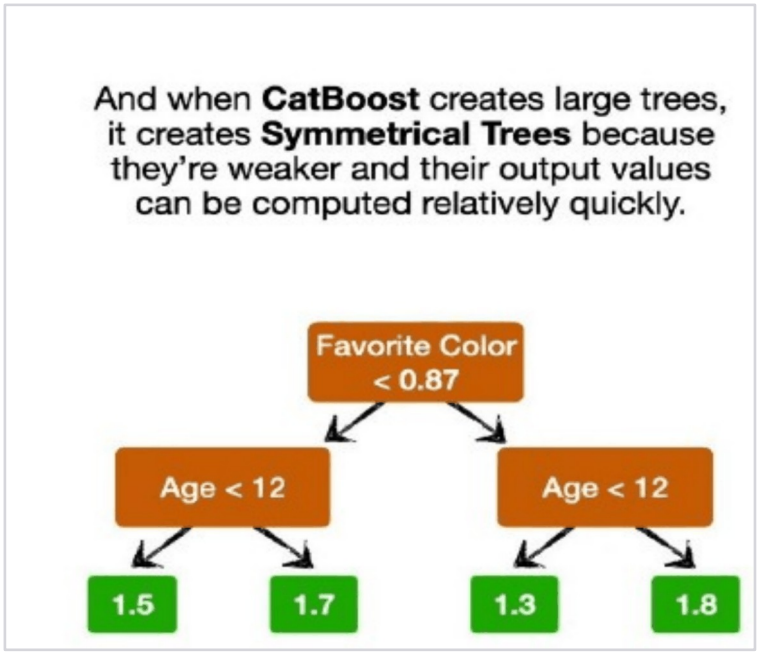
CatBoost trees use the same split at the same depth

Symmetric (oblivious) trees: weaker learners, but fast to compute

A regular tree can ask a different question on each branch



A symmetric tree asks one question per depth



With question order fixed, one vector operation finds the leaf, and overfitting also falls.

Features with the same information are bundled into one

Ordered Feature Combination

How to read it

- In this example, country alone determines hair color.
- Two features with the same information gain are bundled into one.
- This lightens the burden of selecting features in preprocessing.

country	hair color	class_label
India	black	1
India	black	1
India	black	1
india	black	1
russia	white	0
russia	white	0
russia	white	0
russia	white	0

CatBoost is strongest with many categorical features

Other features and limits: the original summary updated to CatBoost 1.2.10

Item	Details
One-hot encoding	Categories with cardinality up to one_hot_max_size use one-hot instead of target encoding
Default parameters	Well-tuned defaults leave relatively little to gain from tuning
Parameters to check first	Adjust in this order: learning_rate, depth, l2_leaf_reg, random_strength
Limits	Trains slower than LightGBM when most features are numeric
Run results	In this lecture's synthetic-data test, training took about 8–10x as long as the other two engines

04

PART 4 · IMPLEMENTATION

Implement with Claude Code

People set the big picture and criteria; Claude Code does the coding

All three libraries shipped new versions in 2026

State versions in the prompt so Claude Code does not copy old examples

3.4.1
XGBoost · August 2026

4.7.0
LightGBM · July 2026

1.2.10
CatBoost · February 2026

Library	Changes to know
XGBoost	feval went in 3.0; gpu_hist and gpu_id went in 3.1. Choose the GPU with device
LightGBM	In 4.0, the early_stopping_rounds and verbose_eval arguments gave way to callbacks
CatBoost	The API is stable across the 1.2 series. The original code works almost as is

Leave installation to Claude Code too. People only state versions in the prompt.

The implementation has five big steps

People decide the steps and criteria; Claude Code writes the code

① Environment and data

Install pinned versions and split the data only once

80% train, 20% validation

② Train three engines

Train with early stopping on the same split

Built-in categoricals

③ Compare

View RMSE and training time in tables and charts

With validation curves

④ Tune

Refine the best engine with Optuna

Tree count by early stopping

⑤ Interpret

Check feature effects with SHAP

beeswarm and waterfall

Prompt ① covers steps ①–③; Prompt ② covers steps ④–⑤.

Every step goes to Claude Code in the same order

Based on the Claude Code desktop app (Code tab): where, input, output, feedback

Where

Code tab

Open the working folder and place the cursor in the input box

e.g., the gbm-lab folder

Input

Prompt

Paste the prompt from the amber box as is

State versions and criteria

Output

Code and results

Claude Code writes and runs the code and shows the results

Files, tables, charts

Feedback

Review

Point out departures from the cautions and have them fixed

Use the checking prompt

Students do not write code themselves. They give criteria and review results.

One prompt builds the three-engine comparison experiment

Steps ①–③: environment setup, data generation, training and comparison on one split

[Input] Prompt · Claude Code (Code tab)

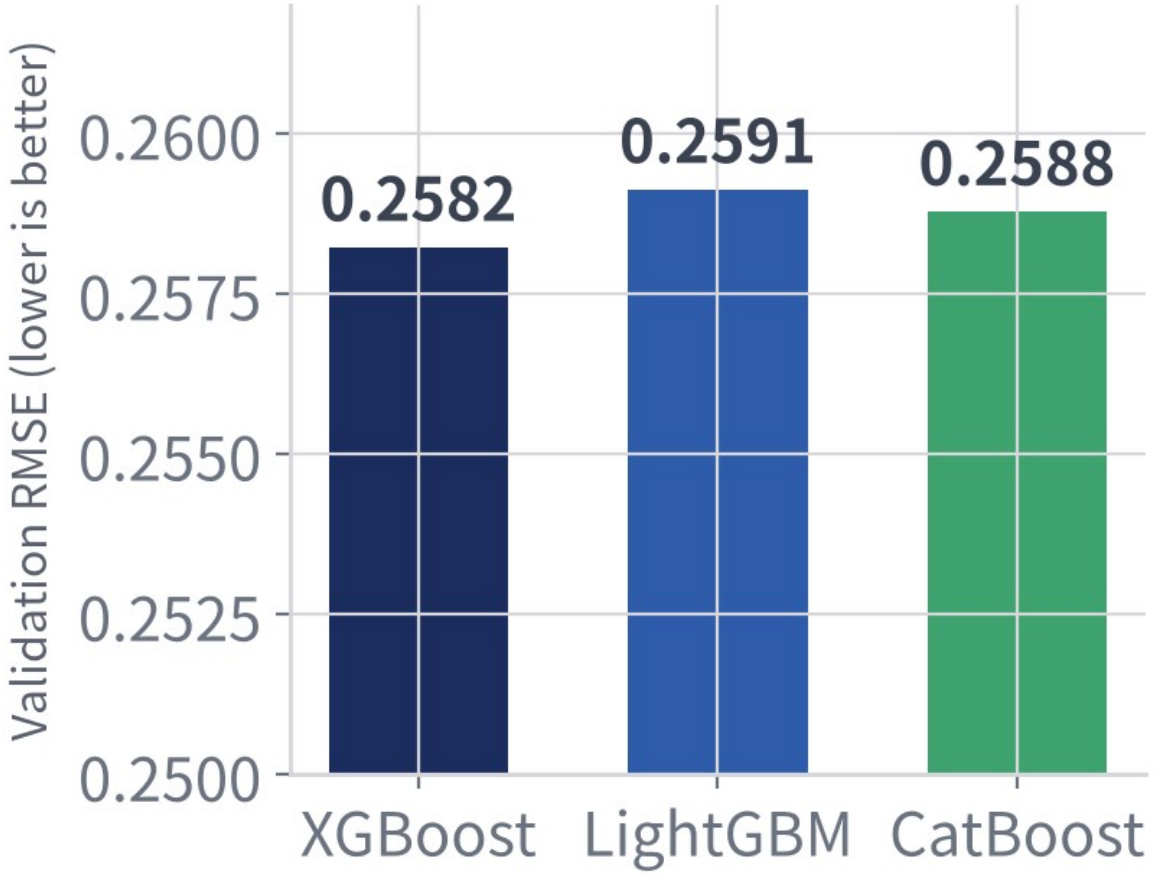
```
In the gbm-lab folder, do the following.
1. Create a virtual environment, install xgboost 3.4,
   lightgbm 4.7, catboost 1.2.10; pin in requirements.txt.
2. Generate synthetic housing-price data with 30,000 rows:
   6 numeric, 2 categorical (region: 60 levels, house_type).
3. Split once into 80% train, 20% validation,
   and have all three engines use the same split.
4. Train the engines with early stopping (100 rounds); save
   validation RMSE, training time and best iteration to results.csv
   and a bar chart. Plot the validation curves too.
5. Handle categoricals with each engine's built-in feature;
   do not use one-hot encoding.
6. Use the 2026 APIs, and tell me about any warnings.
```

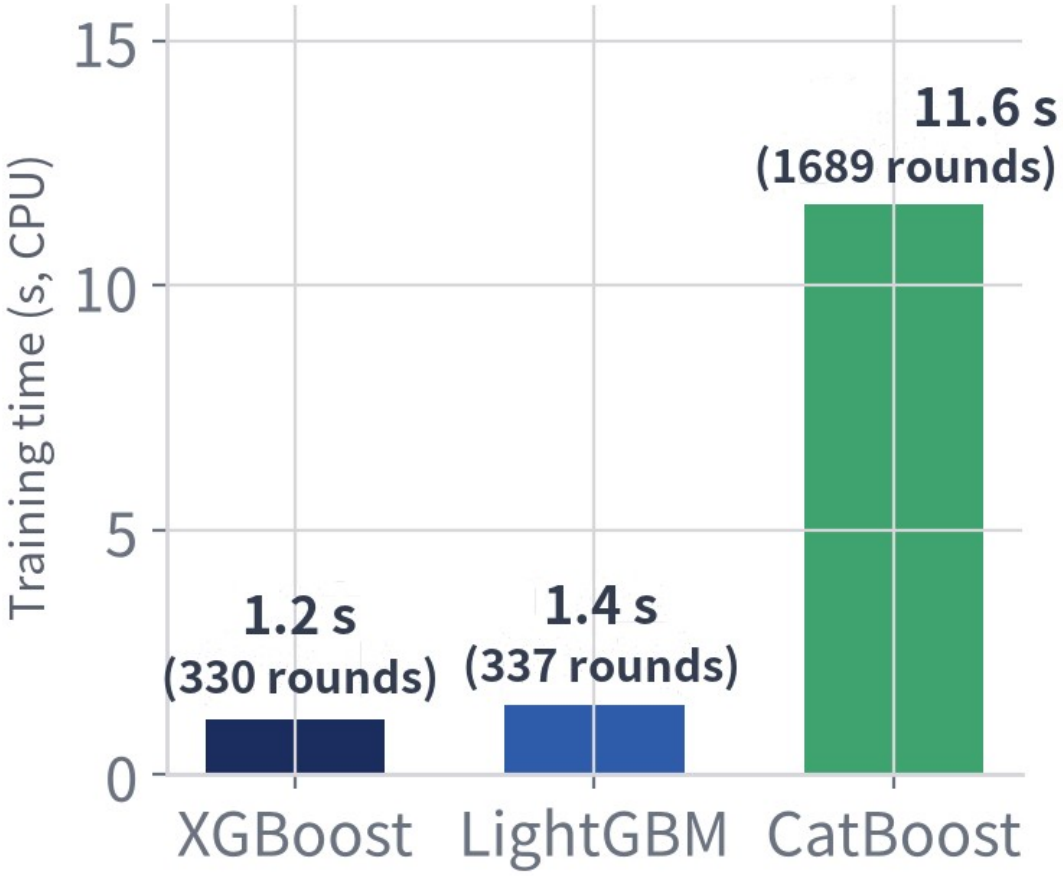
[Output] Results · files Claude Code created

```
requirements.txt
  pinned versions
data.py
  synthetic data generation
train_compare.py
  train and compare three engines
results.csv
  RMSE, time, iterations
bench.png, curves.png
  comparison, validation charts
```

Accuracy differed little across engines on the same data

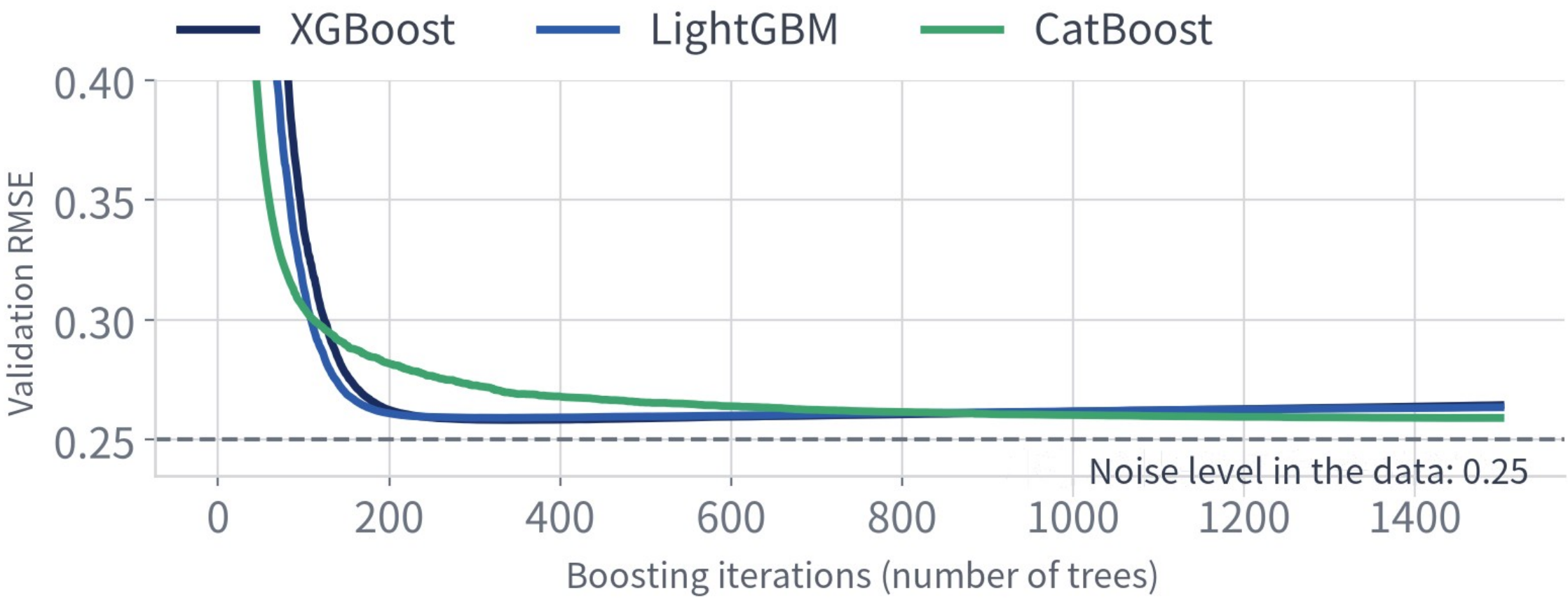
Results of Prompt ①: synthetic data, 30,000 rows, CPU (※ synthetic data)





Early stopping finds the tree count just before overfitting

Validation RMSE curves: XGBoost and LightGBM errors rise again after about 330 rounds



Tuning and interpretation also need only criteria

Steps ④–⑤: Optuna tuning and SHAP interpretation

[Input] Prompt · Claude Code (Code tab)

```
Tune the best engine in results.csv with Optuna.
- 50 trials; objective: minimize validation RMSE
- Search: learning rate (log scale), tree size,
  minimum leaf size, column sampling ratio, L2 penalty
- Do not search the tree count; set it by early stopping
- Compare RMSE before and after tuning in a table

Then run SHAP analysis on the final model.
- A beeswarm plot and a waterfall plot for one row
- Explain the top 5 features' effect direction, a line each
```

What to review

- Was the tree count set by early stopping?
- If tuned on the validation split, was final performance checked on a separate held-out test split?
- Does the SHAP explanation match how the data was generated?

When Claude Code writes old code, correct it with this table

Original lecture code vs. 2026 code, per each library's release notes

Old code	2026 code	Reason
'objective': 'reg:linear'	'reg:squarederror'	Deprecated old name
'silent': 1	'verbosity': 0	silent is no longer used
feval=rmspe_xg	custom_metric=rmspe	feval was removed in 3.0
tree_method='gpu_hist'	device='cuda'	GPU-specific names were removed in 3.1
lgb.train(..., verbose_eval=10, early_stopping_rounds=100)	callbacks=[lgb.early_stopping(100), lgb.log_evaluation(10)]	In 4.0, arguments became callbacks
XGBClassifier().fit(..., early_stopping_rounds=)	XGBClassifier(early_stopping_rounds=)	Moved to the constructor

The same role has different names across engines

Key hyperparameter map: scikit-learn wrapper names; parentheses show aliases or native names

Role	XGBoost	LightGBM	CatBoost
Learning rate	learning_rate (eta)	learning_rate	learning_rate (eta)
Number of trees	n_estimators	n_estimators (num_iterations)	iterations
Tree size	max_depth (default 6)	num_leaves (default 31)	depth (default 6)
Minimum leaf size	min_child_weight	min_child_samples	min_data_in_leaf
Row sampling	subsample	subsample (bagging_fraction)	subsample
Column sampling	colsample_bytree	colsample_bytree (feature_fraction)	rsm
L2 penalty	reg_lambda	reg_lambda (lambda_l2)	l2_leaf_reg
Minimum split gain	gamma	min_split_gain	N/A

When aliases are used together, one is silently ignored

Behavior verified by running: XGBoost 3.4.1, LightGBM 4.7.0, CatBoost 1.2.10

Situation	Actual behavior	Remedy
Give XGBoost both eta and learning_rate	learning_rate value was used with no warning	Use only one of them
Give LightGBM both bagging_fraction and subsample	One warning line, then subsample was ignored	Stick to one naming scheme
Put n_estimators in LightGBM params and also give num_boost_round	params won (20 rounds trained instead of 5)	Set the tree count in one place only
Give CatBoost both learning_rate and eta	An error occurred at training start	Use only one

Write "use only one name per parameter" in the prompt.

Some parameters share a name but differ in meaning

Traps when moving the same values between engines

Parameter	Caution
min_child_weight	In both XGBoost and LightGBM it is a Hessian sum, not a row count. LightGBM's row-count version is min_child_samples (default 20)
max_depth	XGBoost defaults to 6, LightGBM to −1 (no limit). In LightGBM, num_leaves sets tree size
min_data_in_leaf	Per CatBoost docs, used only with Depthwise and Lossguide growth. On the default symmetric trees it passes without error
Random seed	Native names: seed in XGBoost and LightGBM, random_seed in CatBoost. All scikit-learn wrappers accept random_state

Four traps often bite at run time

Behavior actually verified while running this lecture's lab code

Situation	Caution
Missing values in CatBoost categorical columns	NaN raises an error. Fill first with a string such as 'missing'
XGBoost prediction after early stopping	scikit-learn predict stops at the best iteration, but Booster.predict uses all trees unless iteration_range is given
LightGBM tuning	Searching over min_child_samples raises an error. Pass feature_pre_filter=False to Dataset
LightGBM bagging	bagging_fraction works only if bagging_freq is above 0

Finally, have Claude Code check its own work

Feedback step: turn Cautions ①–⑤ into a checklist and paste it as is

[Input] Prompt · Claude Code (Code tab)

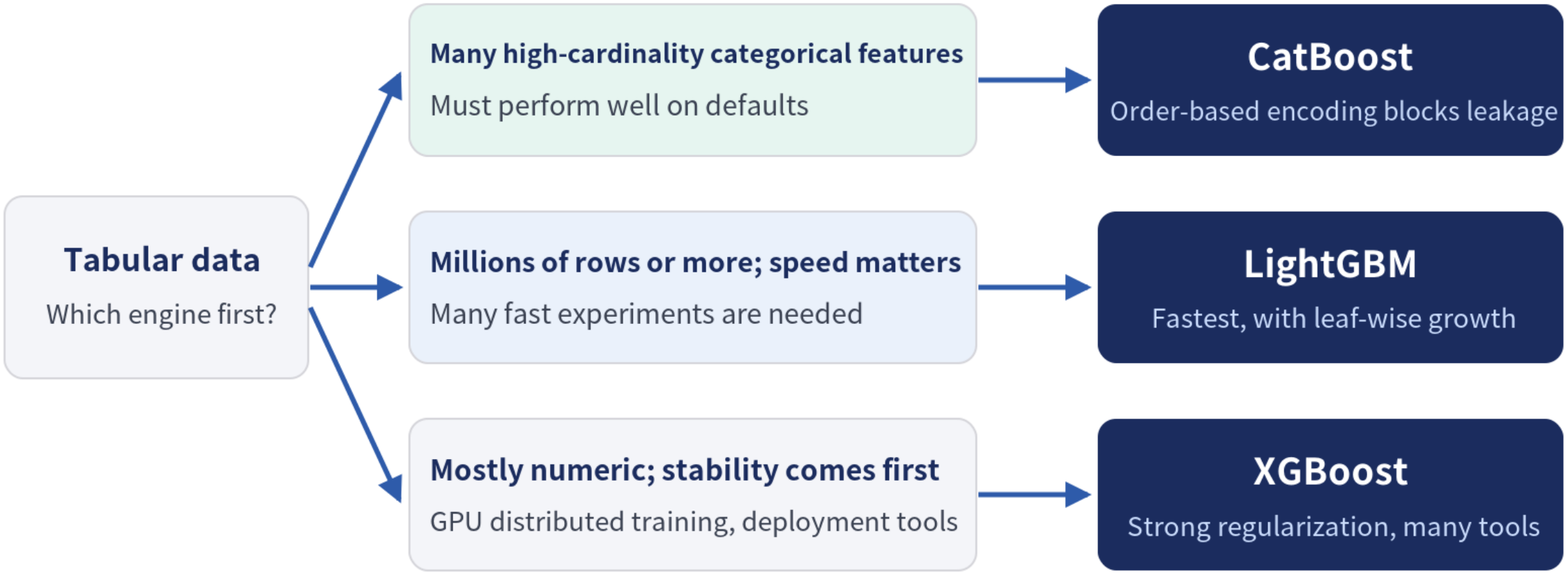
```
Check the code so far against the criteria below;
if there are problems, fix them and tell me what you fixed.
1. Do the installed versions match requirements.txt?
2. Are deprecated names gone (reg:linear, silent, feval, gpu_hist,
   verbose_eval, early_stopping_rounds argument)?
3. Is any parameter given twice through an alias?
4. Did all three engines use the same train/validation split?
5. Did prediction use only up to the best iteration?
6. How were missing values in categorical columns handled?
7. Show me all warnings raised during the run.
```

What to look for in the report

- Does each fix state its reason?
- If warnings remain, ask for the cause again.
- If numbers changed, get the comparison table again.

Choose the engine by data structure and speed needs

A practical starting point; settle the final choice by comparing on the same validation split



On small tables, foundation models are a new rival

TabPFN-2.5 (Prior Labs, November 2025): predicts by in-context learning with no training

Reported results

- Designed for data up to 50,000 rows and 2,000 features.
- Classification win rate against default XGBoost: 100% up to 10,000 rows, 87% up to 100,000 rows.

Where boosting is still strong

- It is fast on data beyond several hundred thousand rows.
- The paper compares against default-setting models; comparison with tuned models needs separate checking.
- Deployment is light and interpretation tools are mature.

GBDT remains the default for large tabular data. On small data, compare TabPFN as well.

Discussion links principles to practical judgment

Answer each question using the formulas and figures of this lecture

Q1	If λ and γ are raised together, how do tree shape and leaf output each change?	**XGBoost regularization**
Q2	With a feature that has many missing values, when is sparsity-aware split finding better than imputation, and when is it risky?	**Missing value handling**
Q3	If num_leaves is raised sharply, what risk grows in LightGBM, and what prevents it?	**Leaf-wise growth**
Q4	How do you prevent leakage when target-encoding a high-cardinality categorical feature?	**Ordered TS**

Three engines: different optimizations, one skeleton

Conclusions of this lecture

1

An XGBoost tree is made with four formulas

The sum and count of residuals, plus λ and γ, set the splits, pruning and output values.

2

LightGBM gains speed by reducing the data to compute

Leaf-wise growth cuts the loss quickly, and GOSS and EFB reduce rows and columns.

3

CatBoost keeps order to prevent leakage

It computes encodings and residuals from earlier rows only, and predicts quickly with symmetric trees.

Leave implementation to Claude Code; people review cautions such as versions and aliases.

A

APPENDIX

Appendix: Paper Algorithms and CatBoost by Hand

The XGBoost paper's algorithms and experiments, and a CatBoost tree followed to the end with a StatQuest example

The paper separates exact greedy from approximate

XGBoost paper, Algorithms 1 and 2 (Chen & Guestrin, 2016)

How to read it

- Exact greedy sorts and scans all split points.
- It cannot be used if data does not fit in memory or in distributed settings.
- The approximate algorithm scans only quantile candidates.

- Split Finding Algorithm
  - ✓ Basic exact greedy algorithm
    - Pros: Always find the optimal split point because it enumerates over all possible splitting points greedily
    - Cons
      - Impossible to efficiently do so when the data does not fit entirely into memory
      - Cannot be done under a distributed setting
  - ✓ Approximate algorithm

Algorithm 1: Exact Greedy Algorithm for Split Finding

Input: I, instance set of current node
Input: d, feature dimension
$gain \leftarrow 0$
$G \leftarrow \sum_{i \in I} g_i$, $H \leftarrow \sum_{i \in I} h_i$
for $k = 1$ *to* m **do**
  $G_L \leftarrow 0$, $H_L \leftarrow 0$
  for j *in sorted*$(I,$ *by* $\mathbf{x}_{jk})$ **do**
    $G_L \leftarrow G_L + g_j$, $H_L \leftarrow H_L + h_j$
    $G_R \leftarrow G - G_L$, $H_R \leftarrow H - H_L$
    $score \leftarrow \max(score, \frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda})$
  end
end
Output: Split with max score

Algorithm 2: Approximate Algorithm for Split Finding

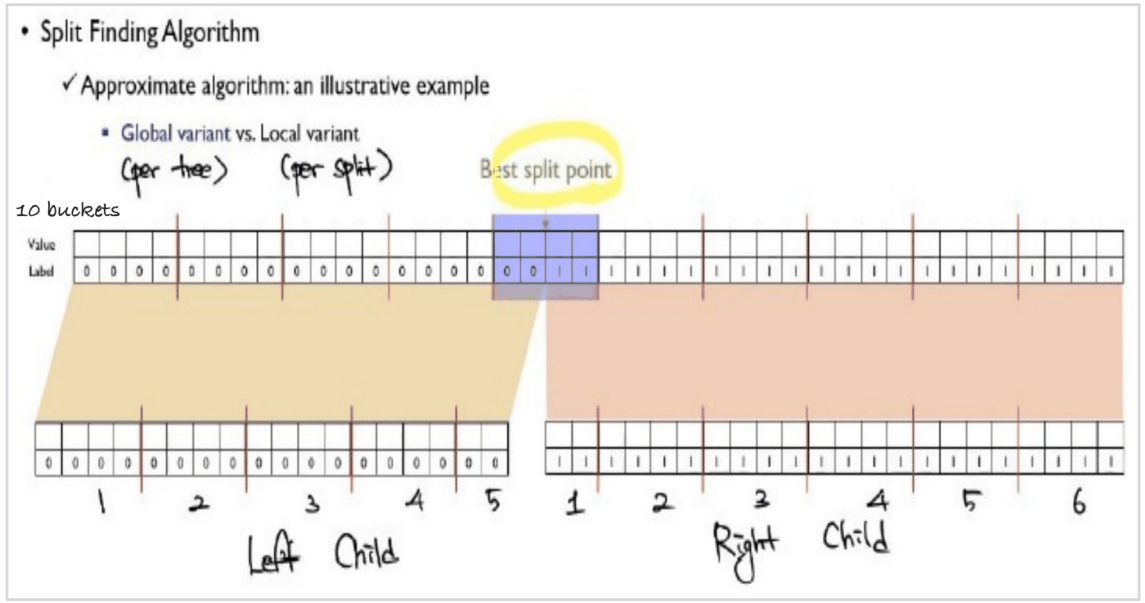
for $k = 1$ *to* m **do**
  Propose $S_k = \{s_{k1}, s_{k2}, \cdots s_{kl}\}$ by percentiles on feature k.
  Proposal can be done per tree (global), or per split(local).
end
for $k = 1$ *to* m **do**
  $G_{kv} \leftarrow= \sum_{j \in \{j | s_{k,v} \geq \mathbf{x}_{jk} > s_{k,v-1}\}} g_j$
  $H_{kv} \leftarrow= \sum_{j \in \{j | s_{k,v} \geq \mathbf{x}_{jk} > s_{k,v-1}\}} h_j$
end
Follow same step as in previous section to find max score only among proposed splits.

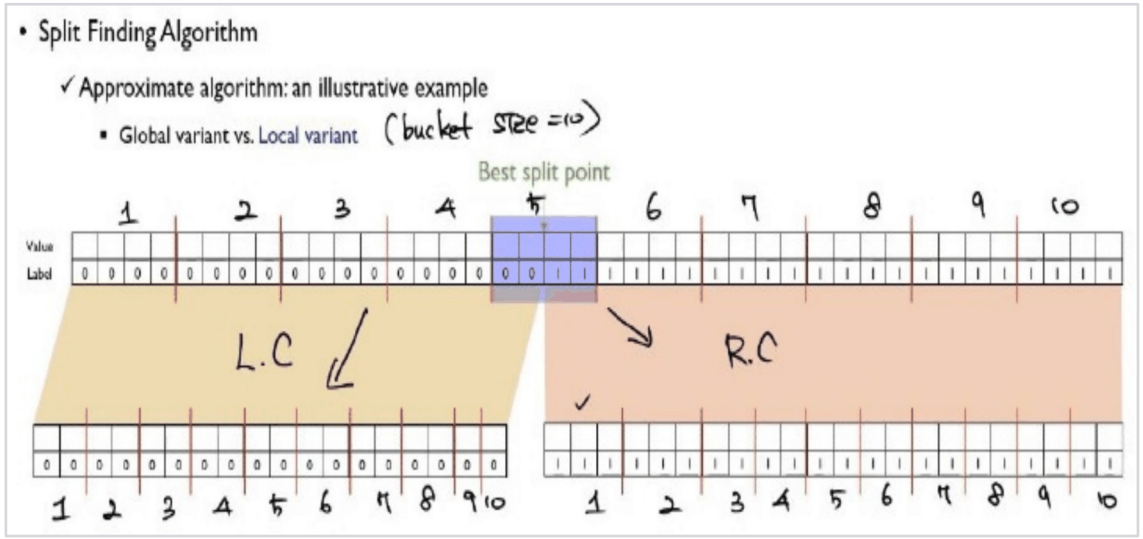
Candidate quantiles are set once or per split

Global and local proposals: hand-drawn explanation from the original

Global: candidates are set once before building the tree

Local: candidates are reset at every split



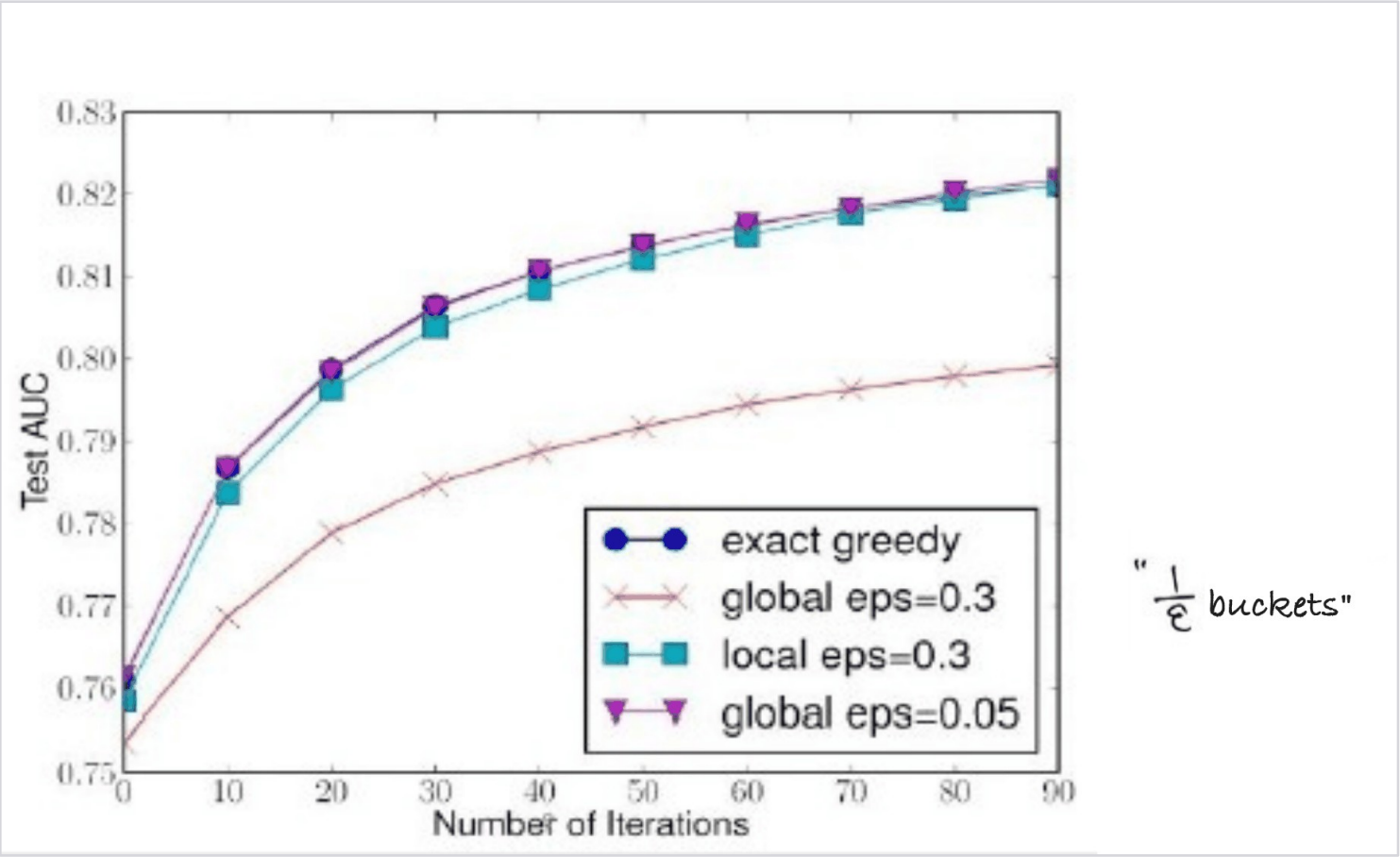


Global proposals need dense candidates to approach local

Paper Figure 3: eps is the quantile spacing; smaller means more candidates

How to read it

- Accuracy drops sharply at global eps 0.3.
- Global eps 0.05 comes close to local eps 0.3.
- More candidates are more accurate but slower.



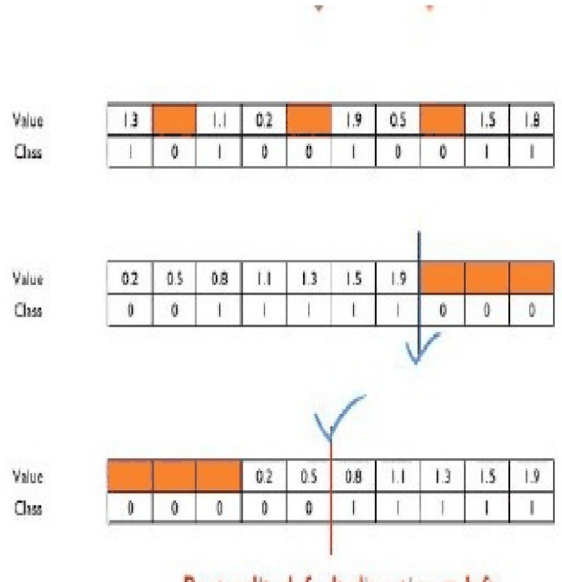
The sparsity-aware algorithm scans only non-missing values

Paper Algorithm 3 and Figure 5: about 50 times faster than the basic algorithm on sparse data

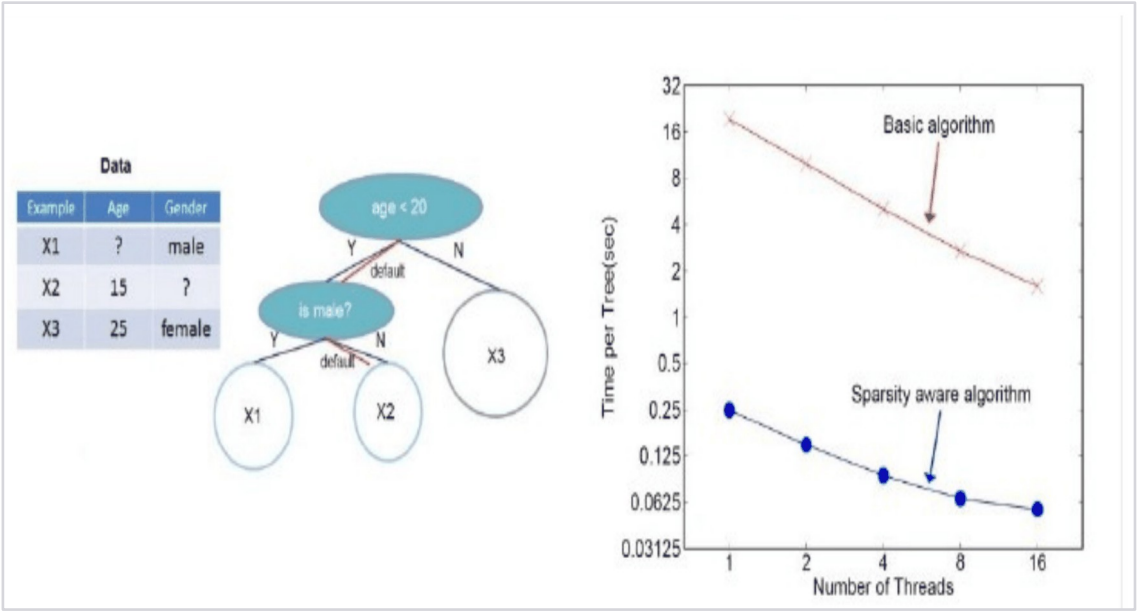
Missing values are sent right and left, in two scans

Algorithm 3: Sparsity-aware Split Finding

Input: I, instance set of current node
Input: $I_k = \{i \in I | x_{ik} \neq \text{missing}\}$
Input: d, feature dimension
Also applies to the approximate setting, only collect statistics of non-missing entries into buckets
$gain \leftarrow 0$
$G \leftarrow \sum_{i \in I} g_i, H \leftarrow \sum_{i \in I} h_i$
for $k = 1$ *to* m **do**
  // enumerate missing value goto right
  $G_L \leftarrow 0,\ H_L \leftarrow 0$
  for j *in sorted*$(I_k,$ *ascent order by* $\mathbf{x}_{jk})$ **do**
    $G_L \leftarrow G_L + g_j,\ H_L \leftarrow H_L + h_j$
    $G_R \leftarrow G - G_L,\ H_R \leftarrow H - H_L$
    $score \leftarrow \max(score, \frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda})$
  end
  // enumerate missing value goto left
  $G_R \leftarrow 0,\ H_R \leftarrow 0$
  for j *in sorted*$(I_k,$ *descent order by* $\mathbf{x}_{jk})$ **do**
    $G_R \leftarrow G_R + g_j,\ H_R \leftarrow H_R + h_j$
    $G_L \leftarrow G - G_R,\ H_L \leftarrow H - H_R$
    $score \leftarrow \max(score, \frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda})$
  end
end
Output: Split and default directions with max gain



Faster than the basic algorithm at any thread count

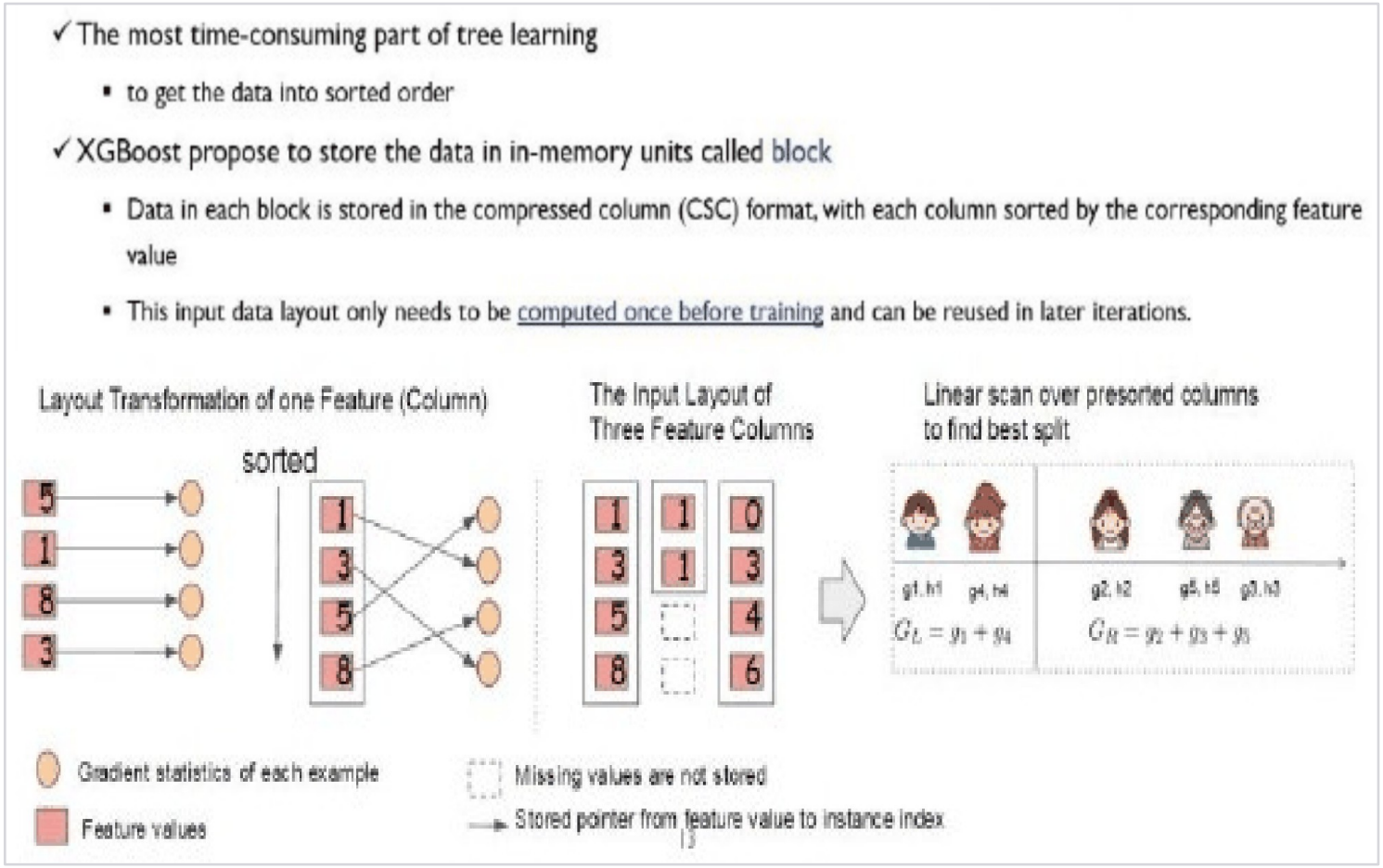


The block structure sorts only once

Paper's System Design: column-wise compressed blocks (CSC)

How to read it

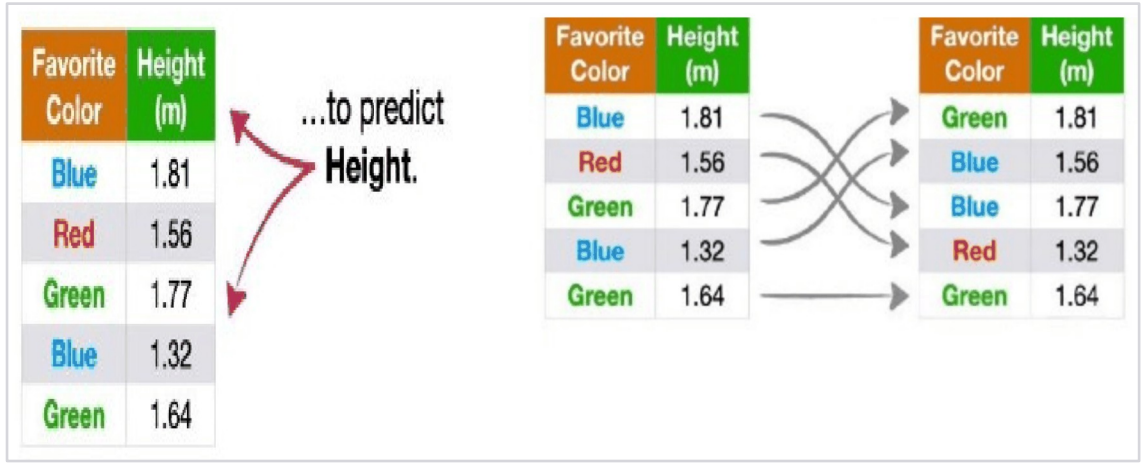
- The most expensive step in tree learning is sorting values.
- Each column is sorted once, stored in blocks and reused every iteration.
- Blocks are compressed and read from disk; another thread decompresses them.



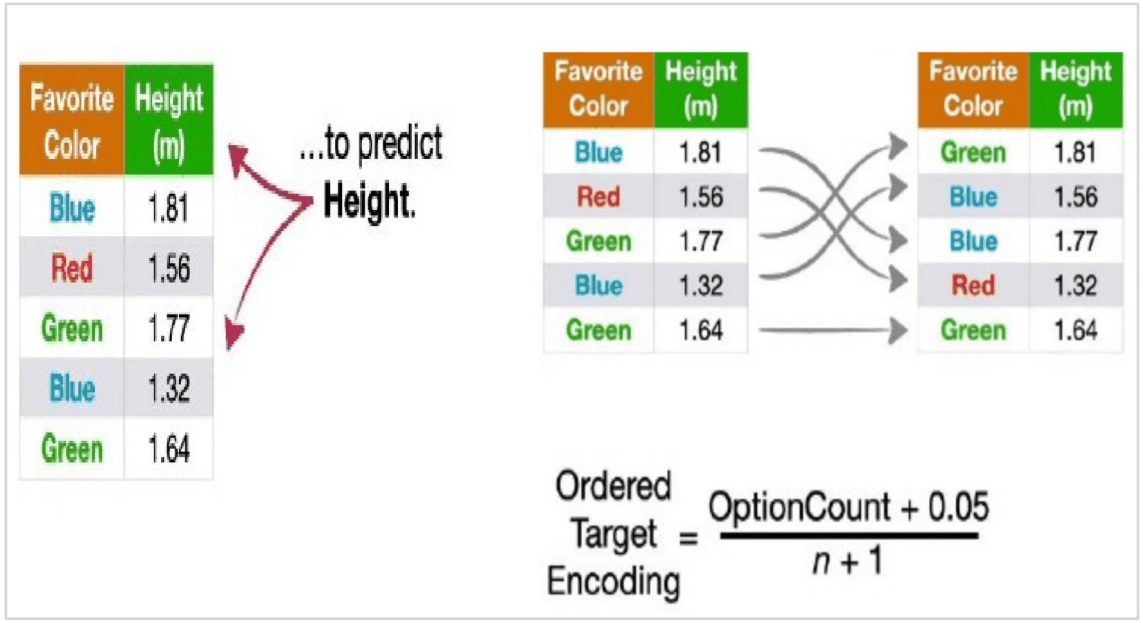
Shuffle rows first, then encode categoricals in order

StatQuest example: a CatBoost tree predicting Height from Favorite Color

Randomly shuffle all rows of the training data



Apply Ordered Target Encoding to the categorical column



This is regression, so Height is cut into two bins and encoded by bin number.

Start at 0, then build residuals and split candidates

Residual = observed – predicted; split candidates lie between sorted encoded values

Predictions start at 0, so residuals equal Height

Favorite Color	Height (m)	Predictions	Residuals
0.05	1.81	0	
0.05	1.56	0	
0.025	1.77	0	
0.05	1.32	0	
0.525	1.64	0	

Residual = Observed - Predicted

Favorite Color	Height (m)	Predictions	Residuals
0.05	1.81	0	1.81

Residual = Observed - Predicted

Residual = 1.81 - 0 = 1.81

Favorite Color	Height (m)	Predictions	Residuals
0.05	1.81	0	1.81
0.05	1.56	0	1.56
0.025	1.77	0	1.77
0.05	1.32	0	1.32
0.525	1.64	0	1.64

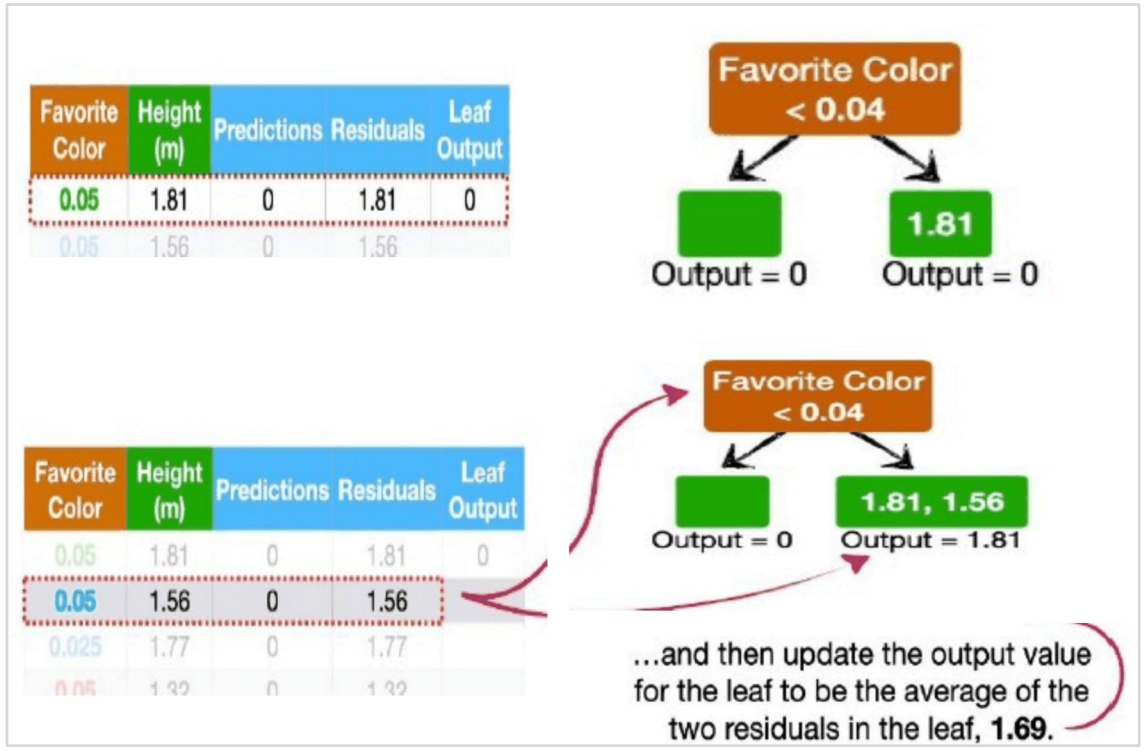
The candidate thresholds are 0.04 and 0.29



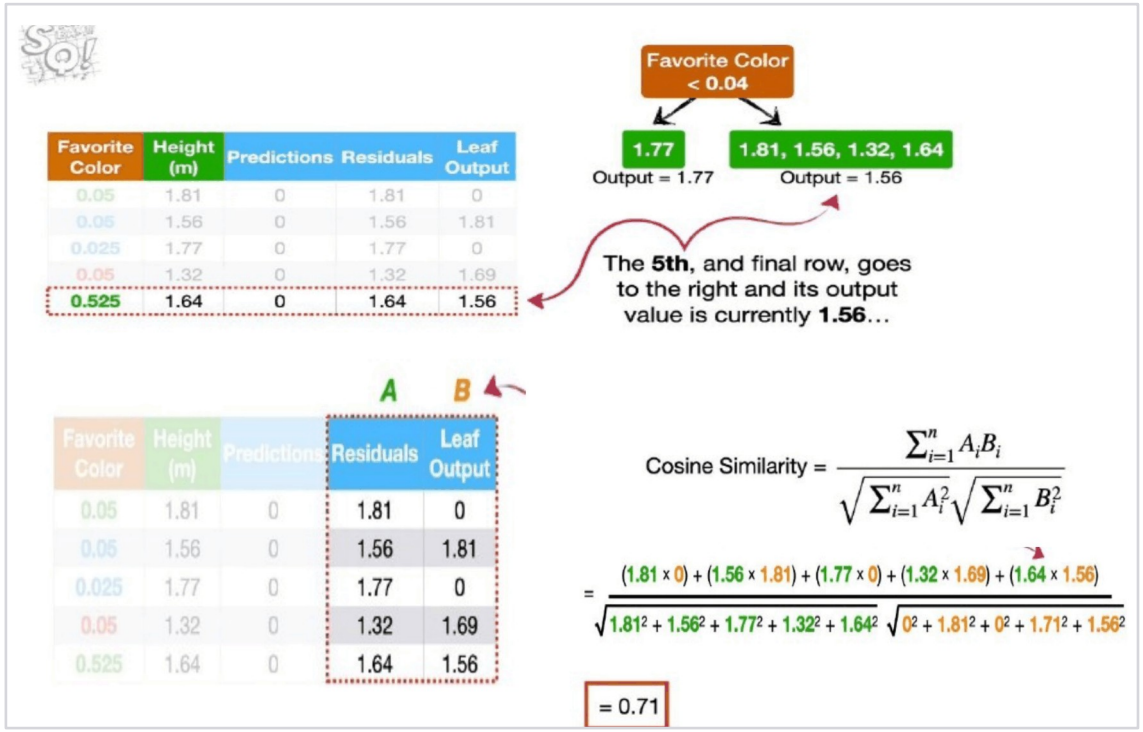
Leaf outputs use only residuals of earlier rows

Go down row by row, record leaf outputs, and find the cosine similarity

A row's own residual is not included in that row's leaf output



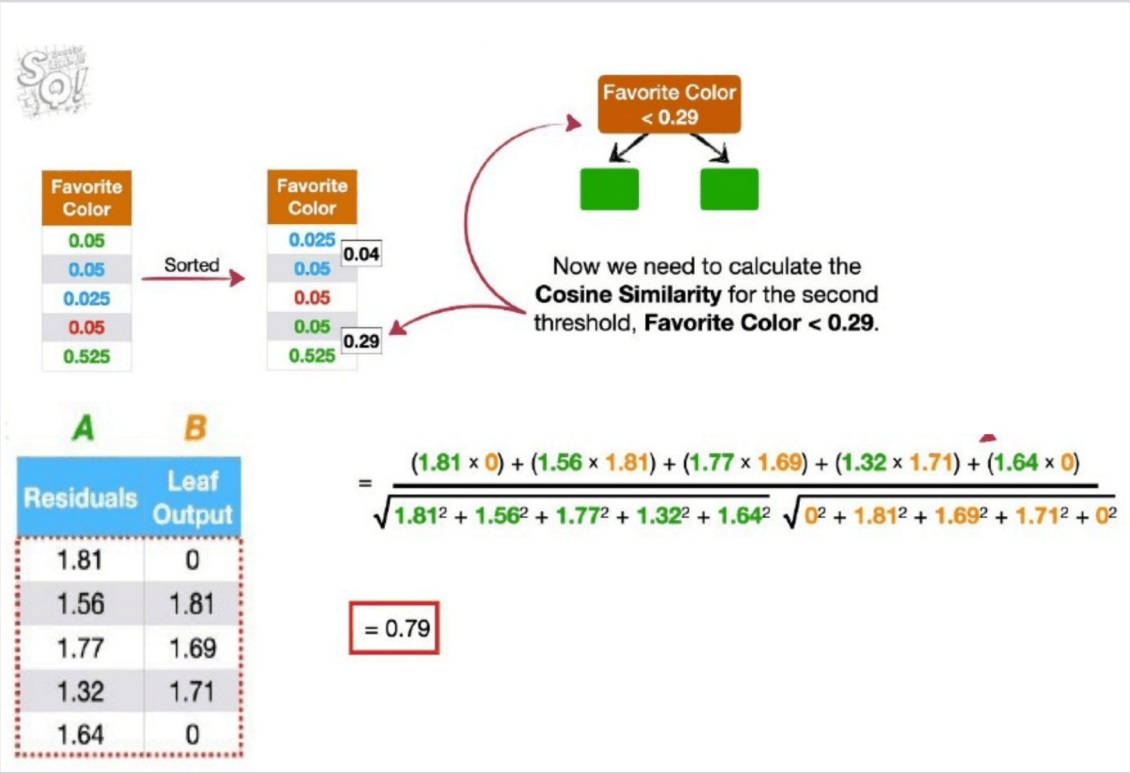
The cosine similarity of threshold 0.04 is 0.71



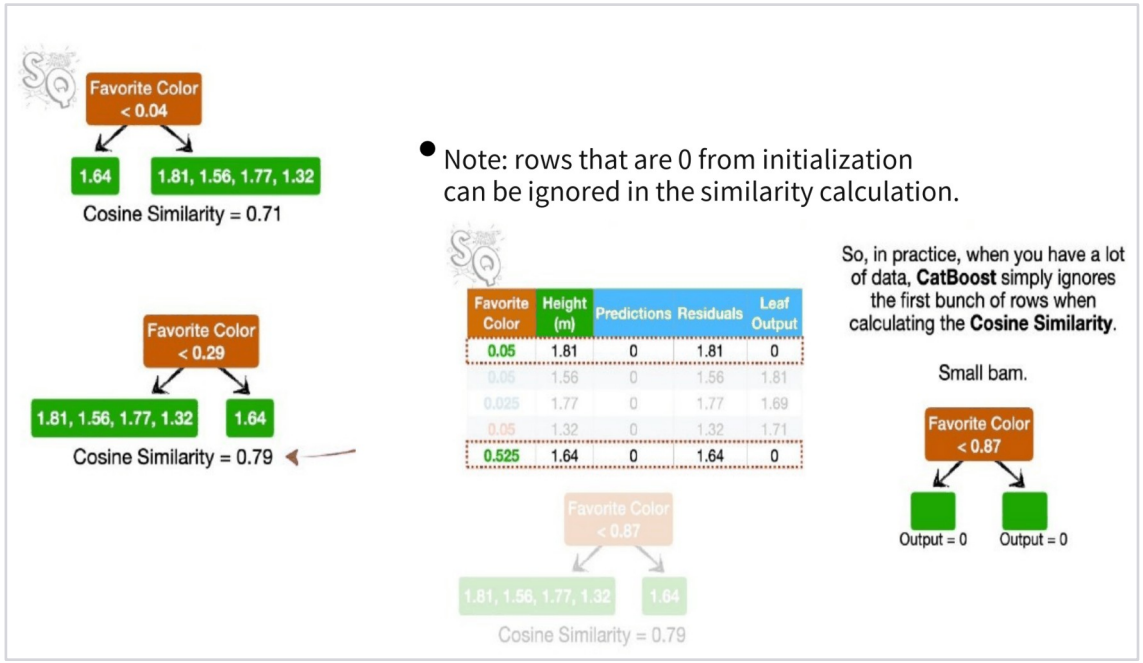
Choose threshold 0.29, which has higher similarity

Apply the same calculation to threshold 0.29

The cosine similarity of threshold 0.29 is 0.79



0.79 beats 0.71, so 0.29 is chosen



Update predictions and compute new residuals

New prediction = old prediction + learning rate 0.1 × leaf output

Update predictions with learning rate 0.1

Favorite Color	Height (m)	Predictions	Residuals	Leaf Output
0.05	1.81	0		
0.05	1.56	0.18		
0.025	1.77	0.17		
0.05	1.32	0.17		
0.525	1.64	0		0

Learning Rate = 0.1

New Prediction = Prediction + (Learning Rate × Leaf Output)

New Prediction = 0 + (0.1 × 0)

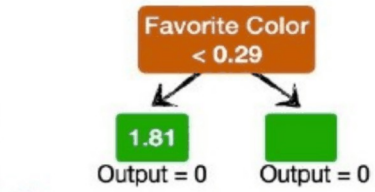
New Prediction = 0.0

Favorite Color	Height (m)	Predictions	Residuals	Leaf Output
0.05	1.81	0		
0.05	1.56	0.18		
0.025	1.77	0.17		
0.05	1.32	0.17		
0.525	1.64	0		

The **Main Idea** of how **CatBoost** calculates the output values for trees is that, just like for **Ordered Target Encoding**, it treats the data as if it is receiving it sequentially.

Residuals come from leaf outputs that never saw that row

Favorite Color	Height (m)	Predictions	Residuals	Leaf Output
0.05	1.81	0	1.81	0
0.05	1.56	0.18	1.56	1.81
0.025	1.77	0.17	1.77	0
0.05	1.32	0.17	1.32	1.69
0.525	1.64	0	1.64	1.56



This means that the **Residual** in a row is not part of the calculation of the **Leaf Output** for that same row…

Favorite Color	Height (m)	Predictions	Residuals	Leaf Output
0.05	1.81	0	1.81	
0.05	1.56	0.18	1.38	
0.025	1.77	0.17	1.6	
0.05	1.32	0.17	1.15	
0.525	1.64	0		

Residual = Observed - Predicted

The next tree reshuffles rows and encodes anew

Order changes with each tree, so order effects average out

How to read it

- Turn encoded values back into the original color names.
- Reshuffle the rows and redo Ordered Target Encoding.
- Choose the split and update leaf outputs the same way.

Favorite Color	Height (m)	Predictions	Residuals	Leaf Output
Green	1.81	0	1.81	
Blue	1.56	0.18	1.38	
Blue	1.77	0.17	1.6	
Red	1.32	0.17	1.15	
Green	1.64	0	1.64	

- Next, randomize as before and apply Ordered Target Encoding.

Favorite Color	Height (m)	Bin #	Predictions	Residuals	Leaf Output
Red	1.32	0	0.17	1.15	
Green	1.64	1	0	1.64	
Green	1.81	1	0	1.81	
Blue	1.77	1	0.17	1.6	
Blue	1.56	0	0.18	1.38	

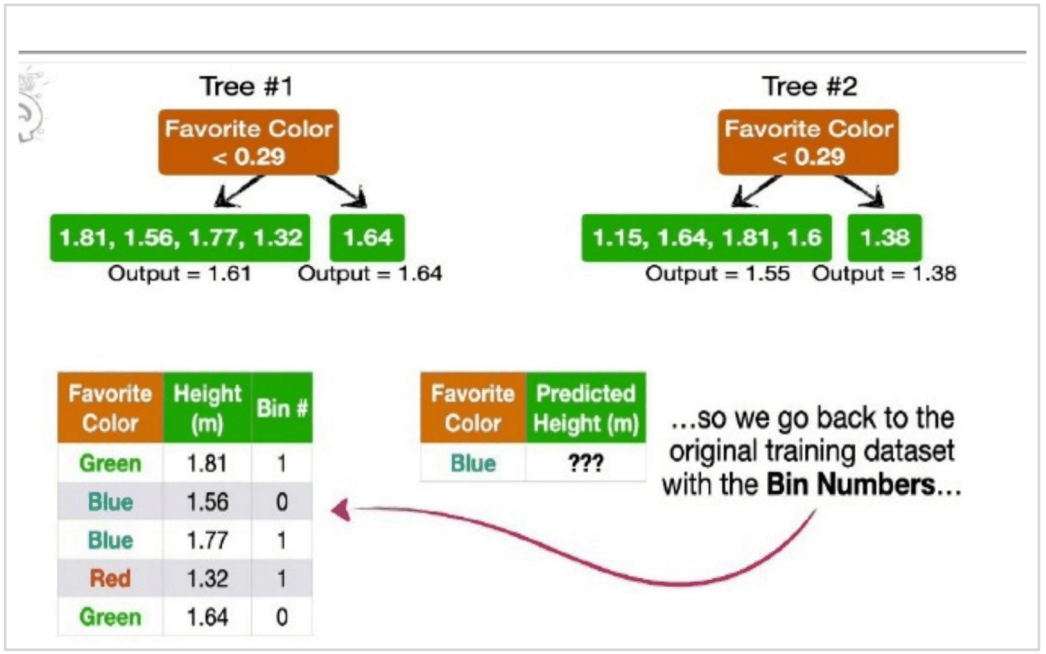


Favorite Color	Height (m)	Bin #	Predictions	Residuals	Leaf Output
0.05	1.32	0	0.17	1.15	
0.05	1.64	1	0	1.64	
0.525	1.81	1	0	1.81	
0.05	1.77	1	0.17	1.6	
0.525	1.56	0	0.18	1.38	

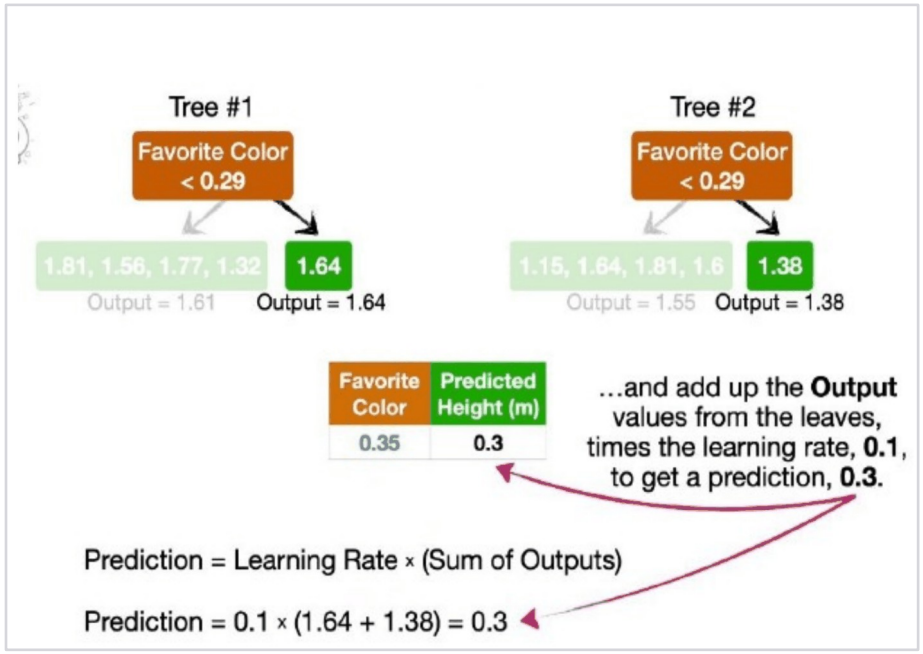
A new row passes through both trees to get a prediction

Prediction = learning rate × sum of leaf outputs = 0.1 × (1.64 + 1.38) = 0.3

Two trees have been built



Encode the new row Blue as 0.35 and feed it to both trees



With more data, more features and bigger trees, predictions improve.